

Department of Artificial Intelligence Engineering

LG전자 KAMP 교육과정(키친솔루션 & 빌트인 쿠킹솔루션)

로컬 환경에서의 챗봇 Agent 구현 및 멀티 Agent 구현 실습

Contents

목차

- 01 — 로컬 Agent 통합 아키텍처
- 02 — CLI 기반 로컬 챗봇 Skeleton 구현
- 03 — 업무 Tool 구현과 Tool Registry 구성
- 04 — 문서 검색 Tool과 Context 주입 구조
- 05 — Single Agent Controller 구현
- 06 — Multi-Agent 소개와 간단한 역할 Workflow 구현

01

로컬 Agent 통합 아키텍처

● 선행 학습 요소

- LLM 기본 개념, prompt engineering
- Agent, RAG, MCP 기본 개념
- Ollama 기반 local LLM 실행 및 agent 구현 요소

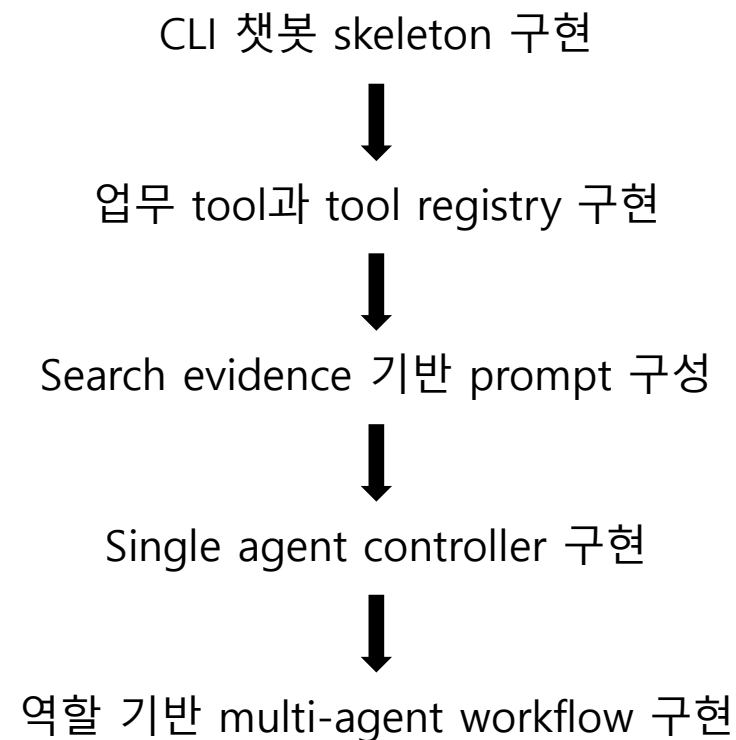
● 오늘의 통합 목표

- CLI 기반 로컬 챗봇 구현
- 업무 tool과 search evidence를 연결한 single agent 구현
- 역할 분해 기반 Multi-Agent workflow 구현

● 수업 산출물

- CLI 기반 로컬 챗봇
- Tool registry와 single agent workflow
- Multi-agent 업무 자동화 workflow

오늘의 구현 흐름



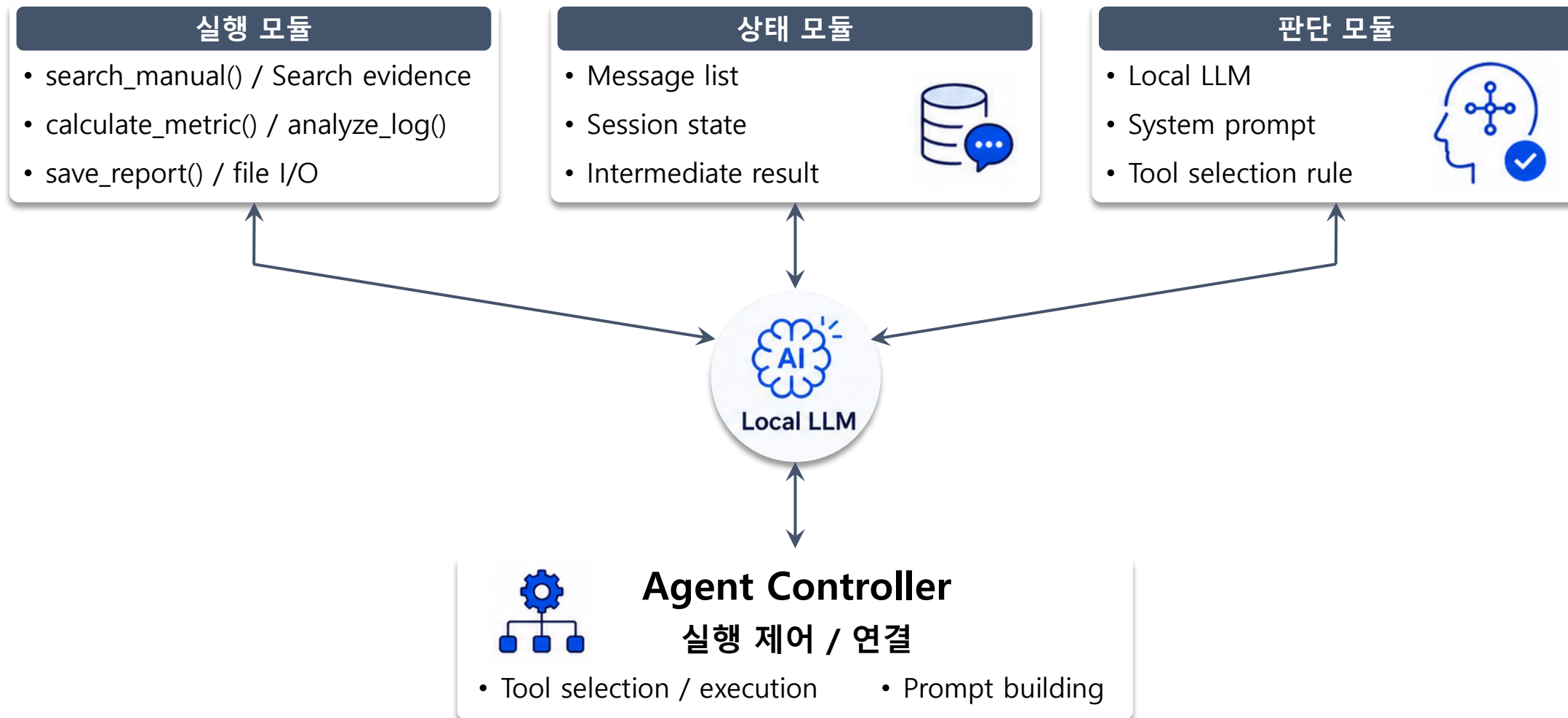
● 개념적 차이

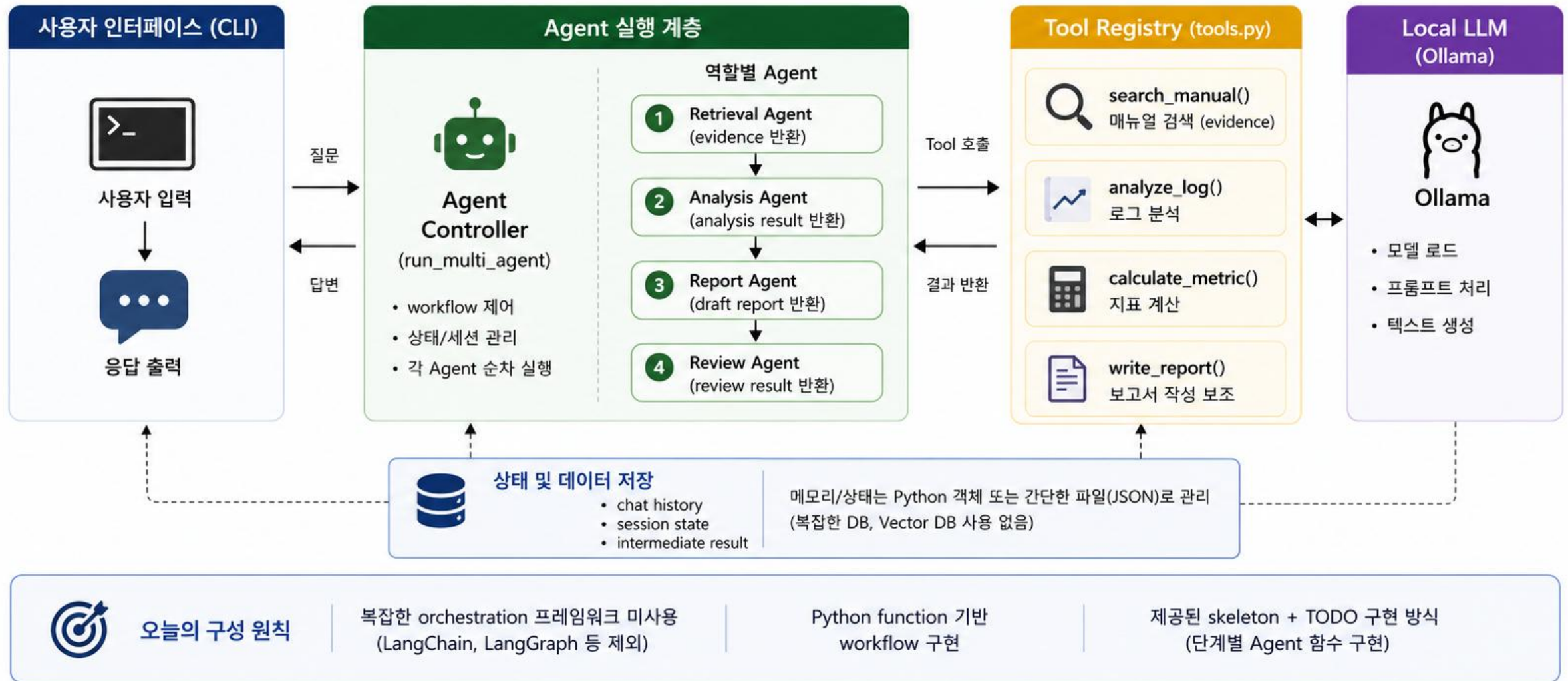
- 챗봇: LLM 응답을 출력
- Agent: 필요한 작업을 판단하고 이를 tool 실행까지 연결

● 구조적 차이



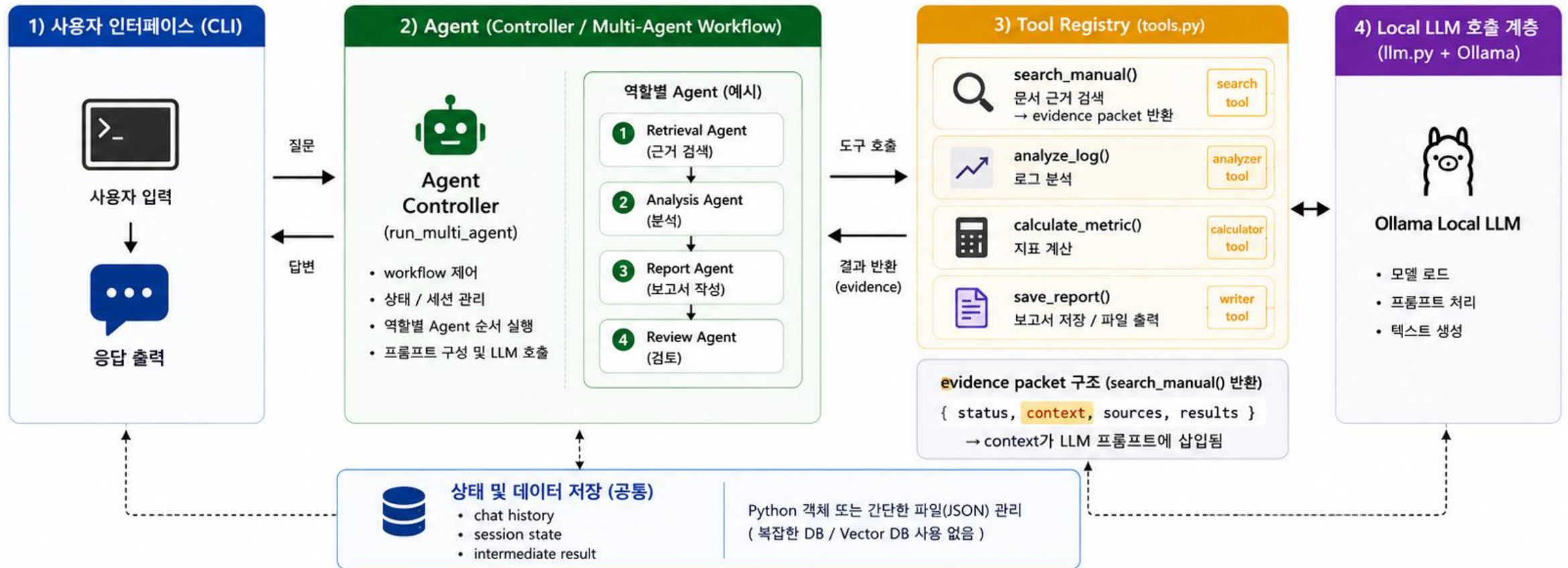
Agent는 LLM을 중심으로 상태, 도구, 검색, 제어 로직이 결합된 시스템





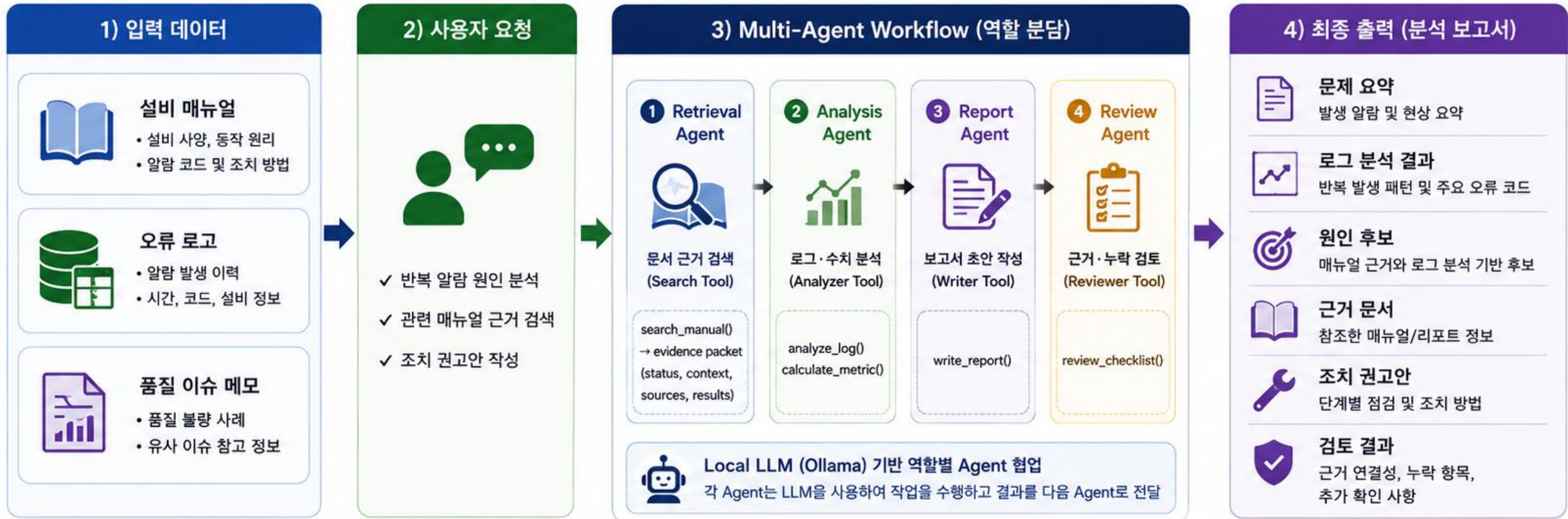
● 개념

- **Tool**: Agent가 실제 작업을 수행하는 실행 단위 (ex. 로그 분석, 산술 계산, 보고서 작성 등)
- **Search**: Tool registry에 등록된 search tool, 실행 결과를 evidence packet으로 반환
→ LLM 프롬프트의 context로 사용



● 목표

- 제조·품질·설비 관련 더미 데이터 기반의 검색-분석-작성-검토 역할을 나눈 Multi-Agent workflow 구현





1) 실행 파일 (CLI Layer)

- chat_cli.py
- 사용자 입력 수신
- run_multi_agent() 호출
- 최종 응답 출력



2) Multi-Agent 파일 (Workflow Layer)

- multi_agent.py
- 역할별 Agent 함수
- run_multi_agent() workflow 제어



3) Tool 파일 (Tool Layer)

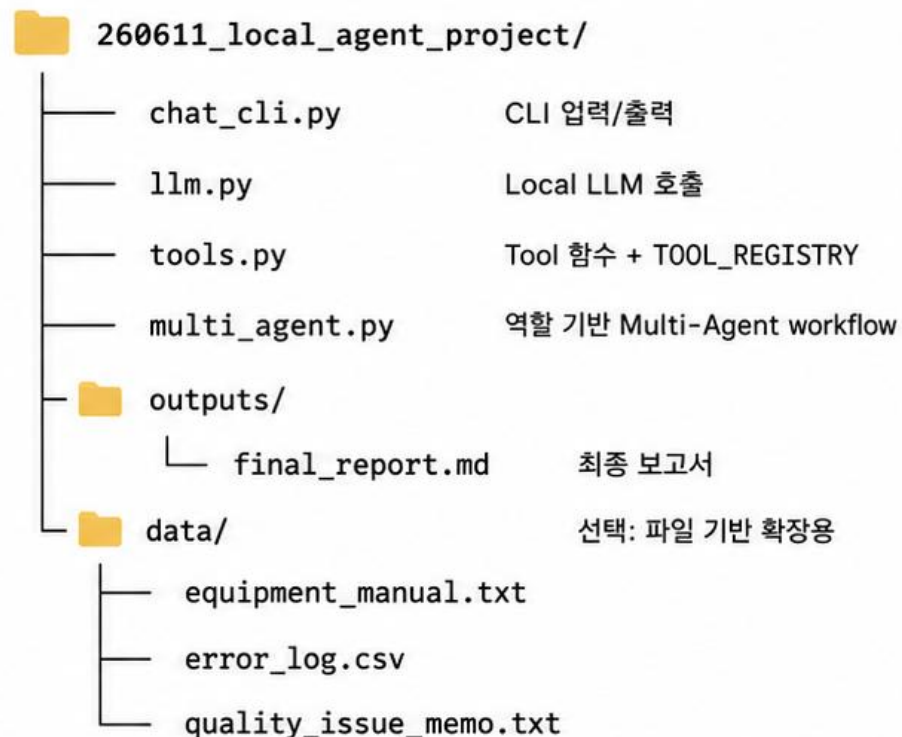
- tools.py
- search_manual()
- analyze_log()
- calculate_metric()
- save_report()
- TOOL_REGISTRY



4) LLM 호출 파일

- llm.py
- Ollama 기반 local LLM 호출
- prompt 전달 및 응답 반환

프로젝트 폴더 구조 (예시)



5) 데이터 (선택 / 확장용)

- Agent가 활용할 업무 데이터
- 실습에서는 sample data로 대체 가능



equipment_manual.txt 설비 매뉴얼



error_log.csv 오류 로그 데이터



quality_issue_memo.txt 품질 이슈 메모



6) 출력 (Output)

- Agent workflow가 생성한 최종 결과



final_report.md

- 문제 요약
- 로그 분석 결과
- 원인 후보
- 조치 권고안
- 검토 결과

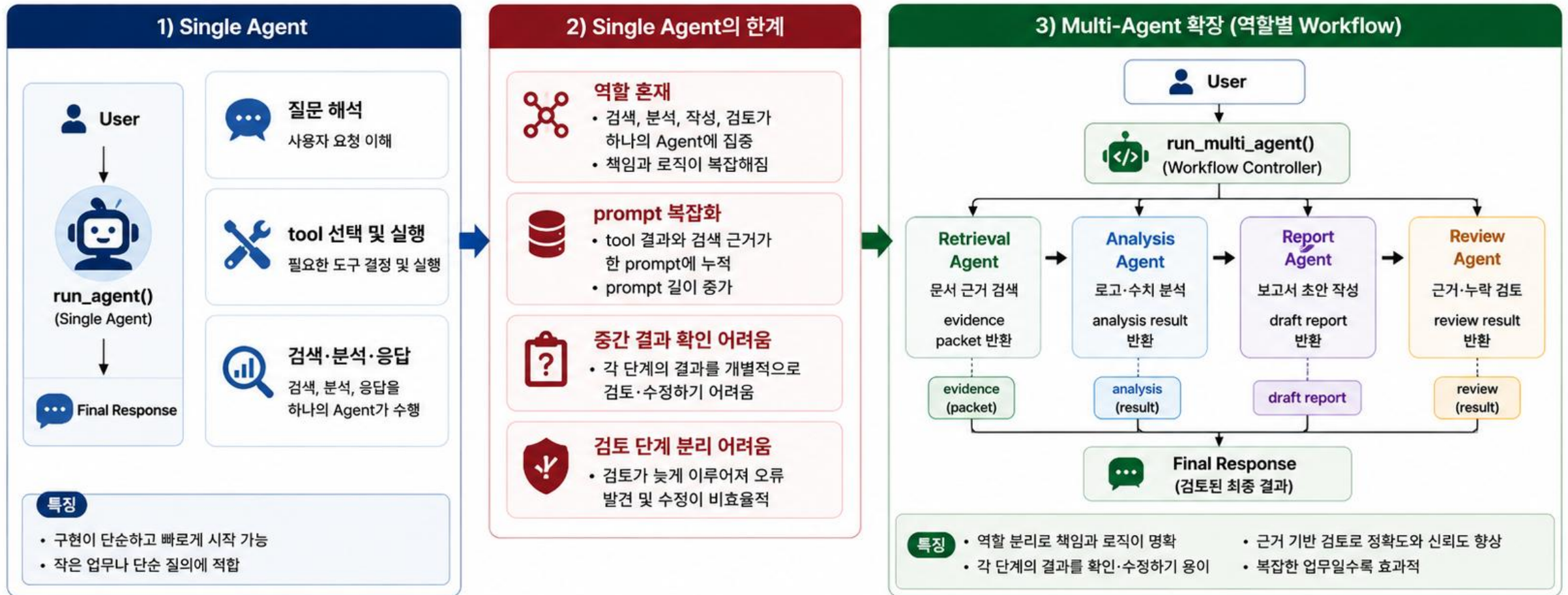


핵심 포인트

파일을 역할별로 분리하면 CLI, tool, LLM 호출, Multi-Agent workflow의 책임이 명확해지고, 각 Agent의 입력과 출력을 단계별로 확인할 수 있다.

● Agent type

- Single agent: 하나의 controller가 전체 업무 흐름을 처리
- Multi-agent: 검색·분석·작성·검토 등 업무 역할별 workflow 분리





1) 기본 구현 (1~2교시)

- 전체 구조 이해
- CLI 기반 대화 환경
- local LLM 호출
- messages list 관리

목표: 로컬 챗봇 skeleton 완성

2) Agent 확장 (3~5교시)

- tool registry 구성
- search_manual() evidence 연결
- tool 선택 · 실행
- run_agent()로 Single Agent 완성

목표: tool + search + LLM을 하나의 Agent 흐름으로 연결

3) Workflow 확장 (6교시)

- 역할별 Agent 소개
- 간단한 순차 workflow 구현
- 중간 산출물 연결 이해

목표: 역할 분리 기반 Multi-Agent workflow 이해

핵심 포인트

- ✓ 1교시는 전체 구조를 이해하고, 2교시부터 실제 코드를 구현한다.
- ✓ 6교시까지 마치면 CLI → Tool → Search → Single Agent → Multi-Agent 흐름을 구현할 수 있다.

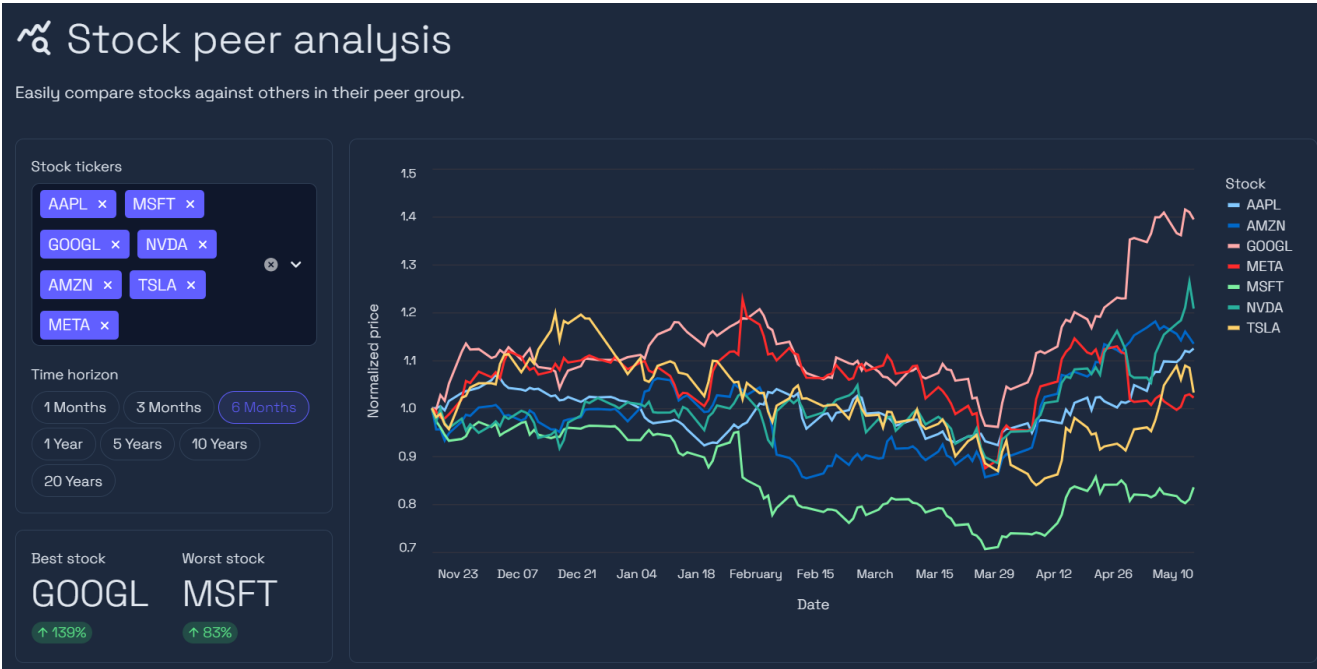
전체 흐름 요약
 Architecture → CLI → Tool → Search → Single Agent → Multi-Agent

02

CLI 기반 로컬 챗봇 Skeleton 구현



- 구현 범위
 - 사용자 입력 수신
 - Local LLM 호출
 - 응답 출력
 - 대화 이력 저장
- 제외 범위
 - Streamlit UI
 - RAG 필수 구현
 - Multi-agent 구현



[Streamlit UI 예시]

● 구현의 단순성

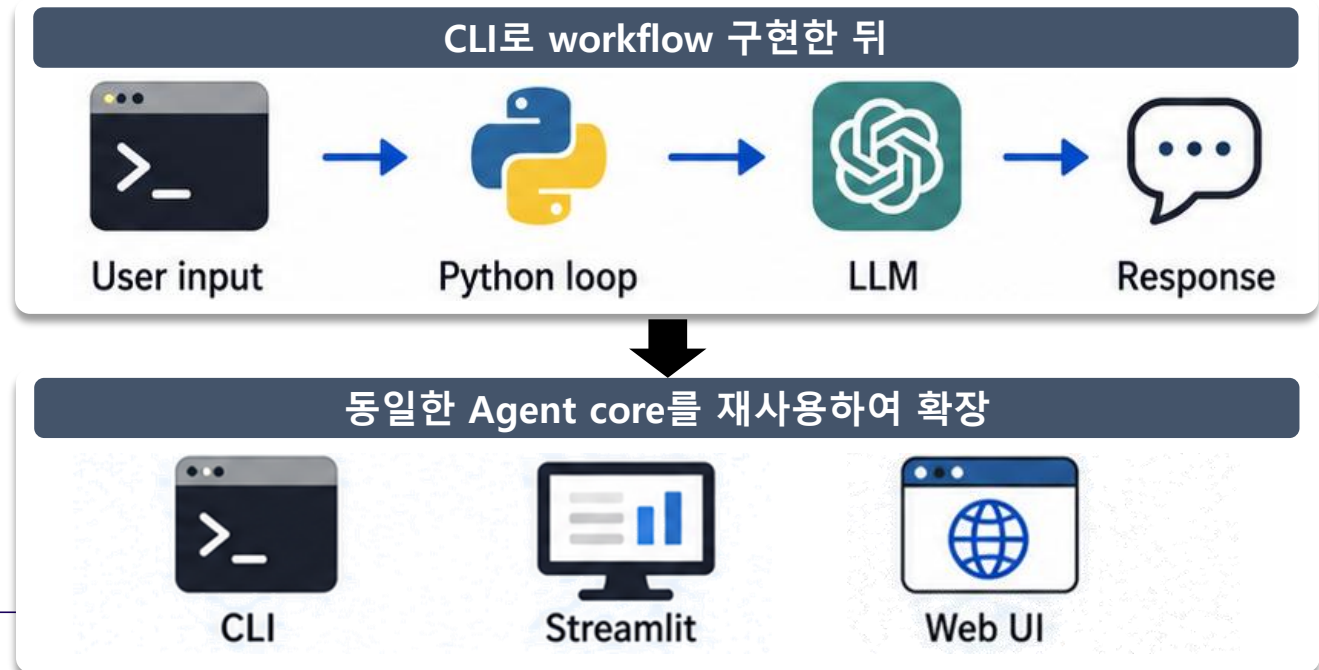
- 별도의 웹 UI 불필요
- Python 실행만으로 테스트 가능
- 오류 원인 파악이 쉬움

● 구조의 명확성

- 입력처리 → LLM 호출 → 응답 출력 → 상태 저장의 구현 흐름이 명확함

● 확장의 용이성

- 필요에 따라 다양한 종류의 tool 연결 가능
- Agent controller로 확장 가능
- 이후 UI layer 교체 가능



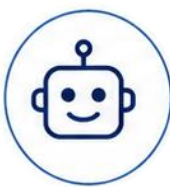
● 챗봇의 기본 구성 요소와 실행 흐름

구성 요소



입력 처리

- 사용자 문장 입력
- 종료 명령 확인
- 빈 입력 방지



응답 생성

- user message 생성
- Local LLM 호출
- Assistant response 반환

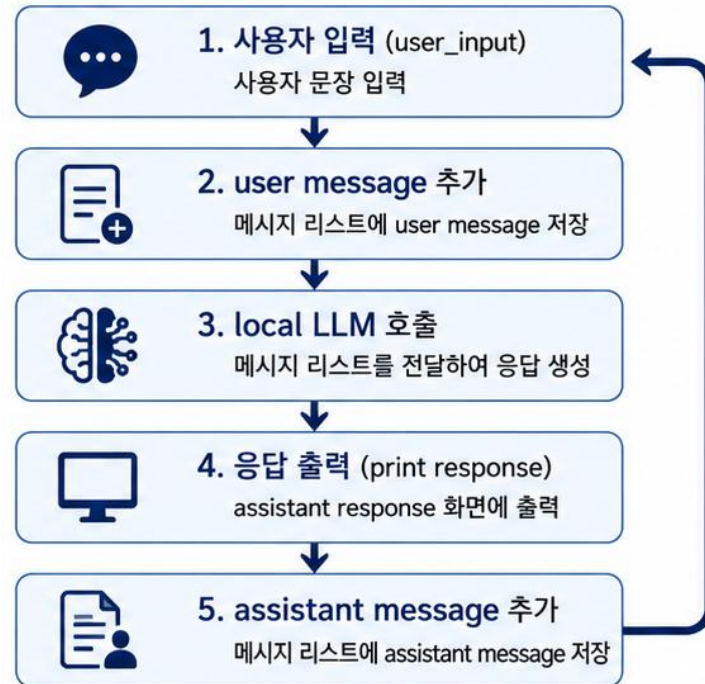


상태 갱신

- user message 저장
- Assistant message 저장
- 다음 질문에서 history 재사용

최소 챗봇 실행 흐름 (반복 루프)

while True



흐름 예시 (의사 코드)

```
while True:
    user_input = input("User: ")
    if user_input in ["exit", "quit"]:
        break

    messages.append({"role": "user",
                     "content": user_input})
    response = call_local_llm(messages)
    print("Assistant:", response)
    messages.append({"role": "assistant",
                     "content": response})
```

- 이후 tool 사용, agent controller, multi-agent 구조로 확장됨
- 이후 각 단계에서 여러 기능이 추가됨

● 맥락 관리의 필요성

➢ 현재 세션의 대화 맥락 유지를 위해 system/user/assistant message를 message list에 누적하여 저장 필요

1

Message List가 필요한 이유

- 이전 질문과 응답을 다음 LLM 호출에 반영
- 후속 질문의 생략된 대상을 해석
- 대화형 응답의 일관성 유지

2

Message 구성

- **system** : 챗봇의 역할과 응답 규칙
- **user** : 사용자 입력
- **assistant** : LLM 응답

3

관리 방식

- Python list 형태로 세션 동안 관리
- 실제 텍스트는 **content**에 저장
- 시간 순서대로 누적
- 프로그램 종료 시 사라짐
- **별도 history 파일은 만들지 않음**

Message List 예시 (Python)

```
# 초기 1회: system prompt 등록
messages = [
    {
        "role": "system",
        "content": "당신은 제조 업무 지원 챗봇입니다.\n답변은 간결하고 구조적으로 작성하세요."
    }
]

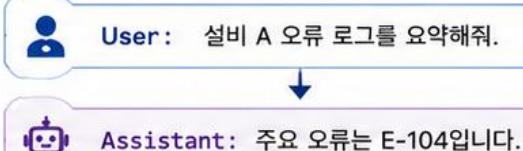
# 이후 반복: user / assistant message 추가
messages.append({"role": "user",
                  "content": "설비 A 오류 로그를 요약해줘."})
messages.append({"role": "assistant",
                  "content": "주요 오류는 E-104입니다."})
messages.append({"role": "user",
                  "content": "그 원인을 설명해줘."})
...

# messages list를 그대로 LLM 호출에 전달
```



세션 내 이전 메시지가 필요한 이유

이전 메시지 반영



현재 질문



이전 메시지가 없으면?
“그”가 무엇을 의미하는지 알 수 없음

Message List가 있으면!
“그” = E-104로 해석하여 정확한 답변 생성

★ 핵심 정리

- ✓ 여기서 말하는 대화 이력은 파일에 저장되는 history가 아니라, **현재 실행 중인 Python 프로그램 안에서 유지되는 messages list**이다.
- ✓ 프로그램을 종료하면 사라지지만, 실행 중에는 이전 user/assistant message가 다음 LLM 호출의 **context**로 사용된다.

! 주의 사항

- 긴 문서, 로그 전체, 검색 결과 전체를 messages list에 계속 누적하지 않는다.
- 필요한 검색 결과, 요약 결과, tool 실행 결과만 현재 질문의 **context**로 전달한다.



● 기본 지시문으로써의 system prompt

- 사용자가 입력하는 질문이 아닌, 개발자가 챗봇의 역할과 응답 규칙을 미리 정해 message list에 첫 번째로 넣는 기본 지시문

1

1. 정의 위치

- chat_cli.py 상단에 SYSTEM_PROMPT로 작성
- messages list의 첫 번째 system message로 등록
- 이후 모든 LLM 호출에 함께 전달

config.py 분리 이후 확장 시 선택 가능

2

2. 정의 내용

- 챗봇의 역할과 목표
- 응답 톤과 형식(구조, 언어 등)
- 모르는 내용은 단정하지 않기
- 근거 부족 시 처리 방식

3

3. 사용자 입력과의 차이

system	전체 대화의 기본 규칙과 역할
user	현재 사용자의 질문
assistant	LLM이 생성한 응답

★ 핵심 정리

- ✓ System prompt는 챗봇의 성격과 답변 규칙을 결정하는 기본 지시문이다.
- ✓ 사용자는 user message만 입력하며, 프로그램은 system prompt와 예전 messages를 함께 LLM에 전달하여 다음 응답의 context를 유지한다.

System Prompt 예시 (chat_cli.py)

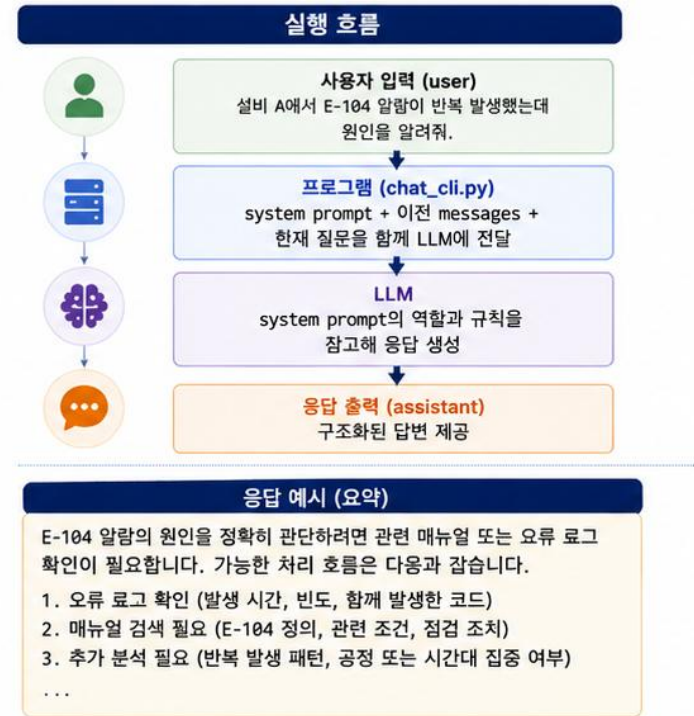
```
# chat_cli.py 상단
SYSTEM_PROMPT = """
당선은 제조 업무 지원 챗봇입니다.

역할:
- 설비, 품질, 오류 로그에 관련된 질문에 답변합니다.
- 답변은 간결하고 구조적으로 작성합니다.
- 모르는 내용은 단정하지 말고, 필요한 추가 정보나 문서
  근거를 요청합니다.
"""
```

메시지 리스트 예시 (Python)

```
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": "E-104 알람 원인을 알려줘."},
    {"role": "assistant", "content": "..."}
]
```

- ▶ messages list는 Python list로 세션 동안 관리됩니다.
- ▶ 프로그램을 종료하면 messages list는 사라집니다.



! 주의 사항

- 긴 문서, 로그 전체, 검색 결과 전체를 messages list에 계속 누적하지 않는다.
- 필요한 검색 결과, 요약 결과, tool 실행 결과만 현재 질문의 context로 전달한다.

💡 핵심

지금은 간단한 규칙만 설정하고, 이후 tool과 search evidence가 붙으면 더욱 중요해진다.



1. 함수 역할

- messages list 입력
- local LLM 호출
- assistant response 반환



2. 분리 이유

- 모델 교체 용이
- prompt 수정 용이
- Agent controller / Multi-Agent workflow에서 재사용 가능



3. 구현 방식

- Ollama 호출 함수 작성
- 오류 발생 시 예외 처리
- 응답 text만 반환



발표 포인트

- ✓ chat_cli.py 안에는 사용자 입력/출력만 두고, LLM 호출은 llm.py의 call_local_llm()로 분리한다.
- ✓ 이 함수는 messages list를 받아 Ollama API를 호출하고, assistant response text만 반환한다.
- ✓ 이렇게 분리하면 이후 agent.py, multi_agent.py 등에서 같은 함수를 재사용할 수 있다.

모듈 분리 구조

chat_cli.py



- 사용자 입력 / 출력 처리
- 사용자 입력 받기
- 응답 출력하기

llm.py



- call_local_llm(messages)
- local LLM (Ollama) 호출
- assistant response 반환

Ollama API



- POST /api/chat 호출
- LLM 추론 수행

assistant response



- 응답 text 반환

예시 코드 (llm.py)

```
import requests

def call_local_llm(messages, model="llama3.1"):
    url = "http://localhost:11434/api/chat"

    try:
        payload = {
            "model": model,
            "messages": messages,
            "stream": False
        }
        response = requests.post(url, json=payload, timeout=120)
        response.raise_for_status()
        result = response.json()
        # Ollama 응답 구조: { "message": { "role": "assistant", "content": "..." } }
        return result["message"]["content"].strip()

    except requests.exceptions.RequestException as e:
        print(f"[LLM 호출 오류] {e}")
        return "최송합니다. LLM 호출 중 오류가 발생했습니다."

    except Exception as e:
        print(f"[예외 발생] {e}")
        return "예기치 않은 오류가 발생했습니다. 다시 시도해주세요."
```

참고

- requests가 없으면 설치 필요

pip install requests

- model은 Ollama에 설치된 모델명과 일치해야 함

예) ollama pull llama3.1



주의 사항

- 긴 문서, 로그 전체, 검색 결과 전체를 messages list에 계속 누적하지 않는다.
- 필요한 검색 결과, 요약 결과, tool 실행 결과만 현재 질문의 context로 전달한다.



핵심

지금은 간단한 규칙만 설정하고, 이후 tool과 search evidence가 붙으면 system prompt가 Agent의 동작 기준으로 더 중요해진다.



CLI chat loop는 사용자 입력, LLM 호출, 응답 출력, **messages list 갱신**을 반복하는 구조이다.



1. 반복 구조

- while True로 무한 반복
- 사용자 입력을 수신
- 종료 명령(exit, quit) 처리



2. 대화 처리

- user message를 messages list에 추가
- messages list를 LLM에 전달
- assistant response 생성
- assistant message를 messages list에 추가



3. 출력 처리

- assistant 응답을 출력
- 오류 발생 시 메시지 출력
- 다음 입력을 기다리며 반복



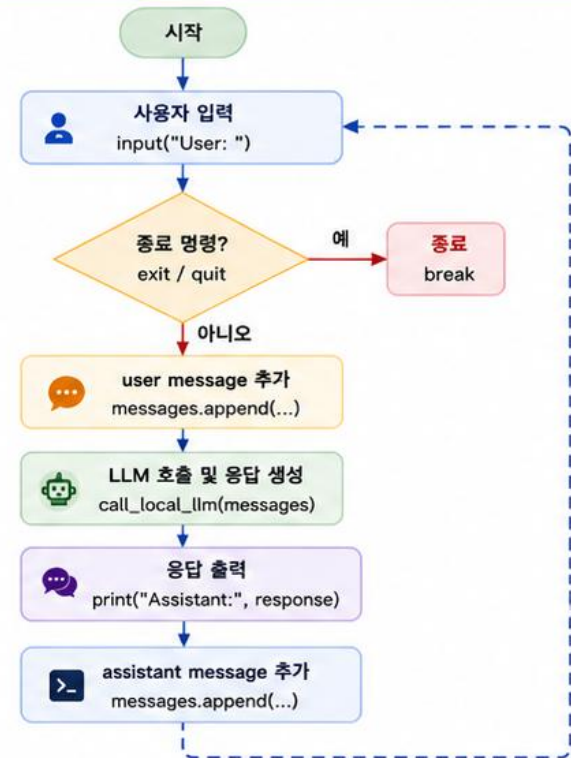
핵심 포인트

- ✓ 이 구조가 이후 Agent loop의 출발점이다.
- ✓ 지금은 단순히 LLM을 호출하지만,
- ✓ 3교시 이후에는 이 위치에 tool 선택, tool 실행, search evidence 구성, 결과 통합 과정이 추가된다.

코드 예시 (chat_cli.py)

```
1 from llm import call_local_llm
2
3 SYSTEM_PROMPT = """
4     당신은 제조 업무 지원 챗봇입니다.
5     답변은 간결하고 구조적으로 작성하세요.
6     모르는 내용은 추정하지 말고,
7     추가 정보가 필요하다고 말하세요.
8 """
9
10 messages = [
11     {"role": "system", "content": SYSTEM_PROMPT}
12 ]
13
14 while True:
15     user_input = input("User: ").strip()
16
17     if user_input.lower() in ["exit", "quit"]:
18         print("종료합니다.")
19         break
20
21     if not user_input:
22         print("빈 입력입니다. 다시 입력하세요.")
23         continue
24
25     # user message 추가
26     messages.append({"role": "user", "content": user_input})
27
28     try:
29         # LLM 호출 및 응답 생성
30         response = call_local_llm(messages)
31
32         # 응답 출력
33         print("Assistant:", response)
34         # assistant message 추가
35         messages.append({"role": "assistant", "content": response})
36     except Exception as e:
37         print(f"[오류 발생] {e}")
```

동작 흐름



이후 확장 흐름 (3교시 이후)



❗ 별도 history 파일을 만들거나 저장하지 않습니다. 실행 중인 프로그램 안에서 **messages list**를 유지합니다.



2교시에서는 파일 구조를 최소화하고, **chat_cli.py**와 **llm.py**만으로 **messages list** 기반 CLI 챗봇을 구현한다.



1. 실행 파일: chat_cli.py

- 사용자 입력 처리
- messages list 관리 (세션 내 대화 맥락 유지)
- 응답 출력



2. LLM 호출 파일: llm.py

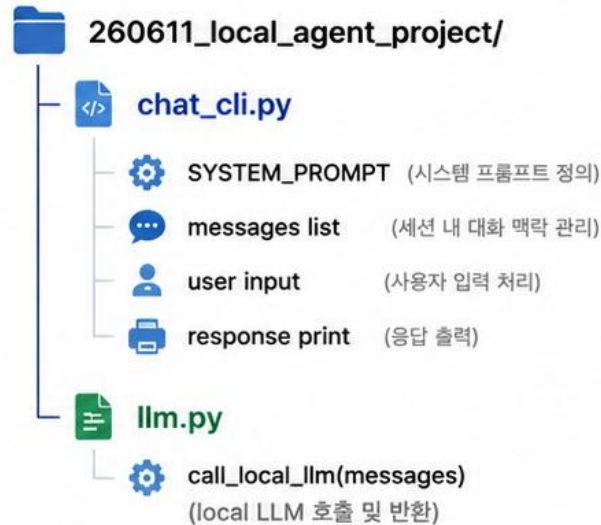
- local LLM(Ollama) 호출
- messages list를 입력으로 받음
- assistant response 반환



3. 단순화 원칙

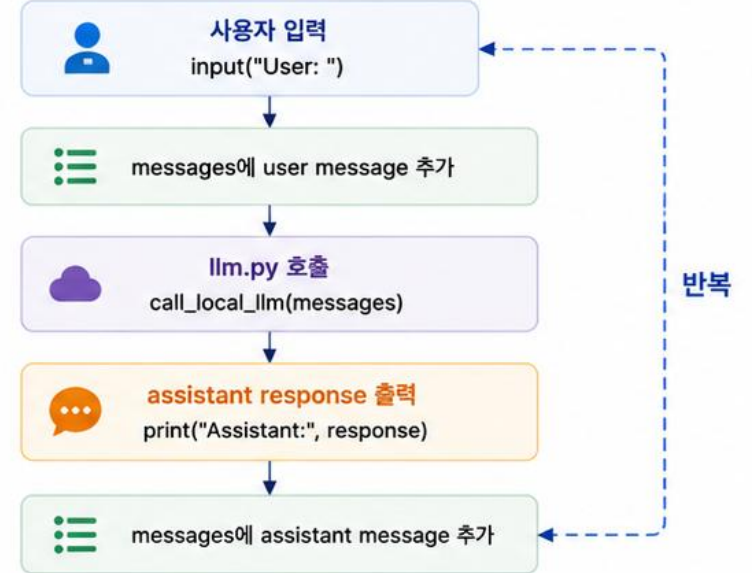
- history 전용 파일은 만들지 않음
- config.py는 필수로 만들지 않음 (선택 사항)
- 이후 교시에서 필요할 때 분리

파일 구조 (최소 구성)



i 이 외의 파일은 만들지 않는다.
(history.py, config.py 등 분리하지 않음)

실행 흐름



하지 않는 것 (이번 교시)

- ✗ history.py 분리
- ✗ state_manager.py 구현
- ✗ config.py 필수 생성
- ✗ memory class 작성
- ✗ DB 저장
- ✗ Streamlit UI 구현



하는 것 (이번 교시)

- ✓ messages list 만들기
- ✓ user/assistant message 누적
- ✓ local LLM 호출
- ✓ 응답 출력



핵심 포인트

챗봇은 **messages list**를 유지하면서 LLM을 반복 호출하는 구조이다.
이 구조가 이후 **Agent loop**의 출발점이다.



이후 확장 흐름 (3교시 이후)



사용자 입력



LLM 호출 (현재 단계)



tool 선택 / 실행 (추가됨)



search evidence 구성 (추가됨)



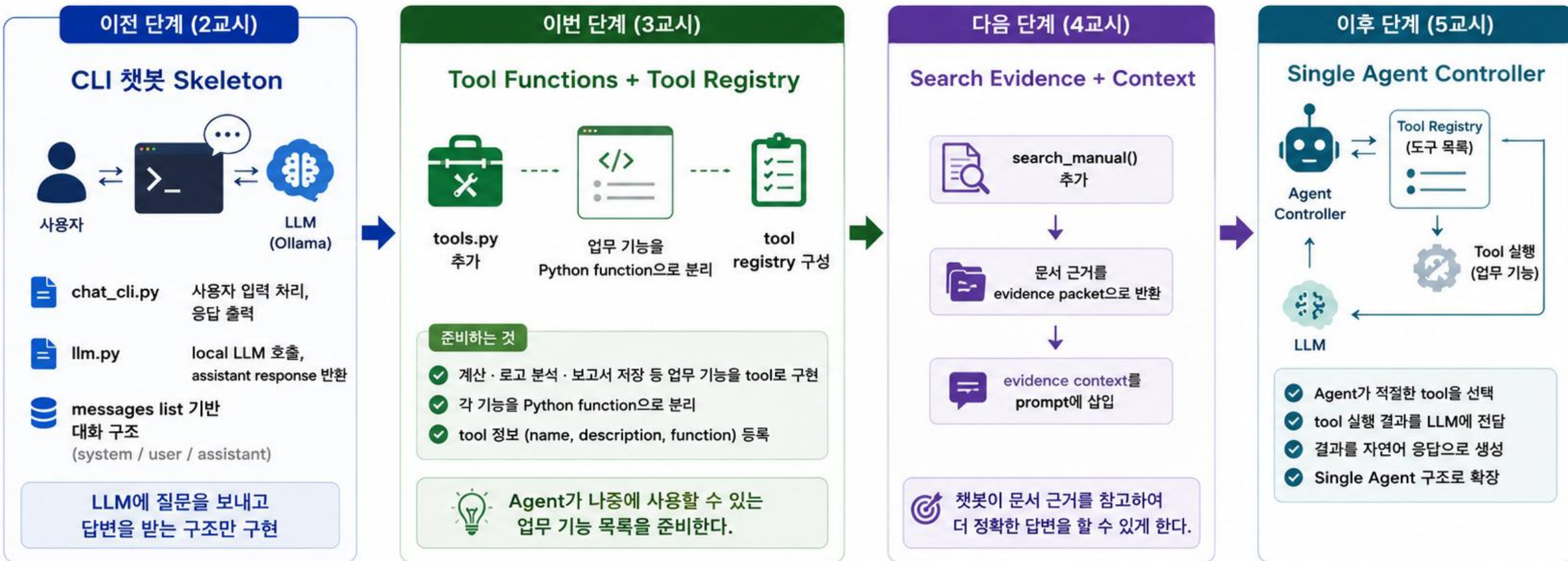
결과 통합 (추가됨)



응답 출력

03

업무 Tool 구현과 Tool Registry 구성



핵심 포인트

- 2교시: LLM과 대화만 하는 구조
- 3교시: 실제 업무 기능을 tool 함수로 만들고, 관리할 준비
- 4교시: 문서 근거를 챗봇 답변에 연결하는 구조

→

- 5교시: Agent가 tool과 evidence를 선택하고 결과를 활용하는 구조

2교시 실습 결과: `chat_cli.py`, `llm.py`

모든 단계에서 `messages list`를 유지하면서 대화를 이어간다.

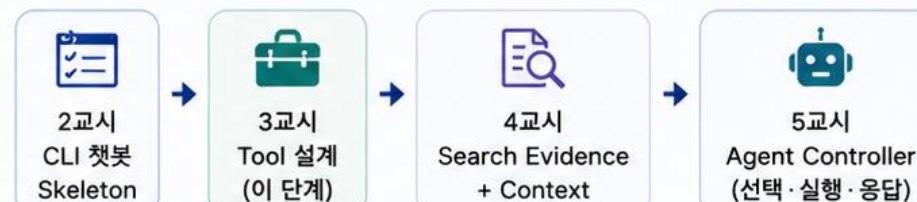
● Tool의 정의

➢ Agent가 사용자 요청을 처리하기 위해 선택 및 실행할 수 있는 외부 기능 단위



업무 목적	Tool 예시	역할
오류 로그 분석	analyze_log()	오류 코드 빈도 · 반복 알람 분석
수치 지표 계산	calculate_metric()	평균, 합계, 비율 계산
보고서 저장	save_report()	분석 결과를 Markdown 파일로 저장
문서 근거 확인	search_manual() ※ 4교시 확장	문서 근거를 evidence packet으로 반환

전체 흐름 요약



● Tool의 별도 구현

- LLM이 직접 처리하기 어려운 계산, 로그 분석, 파일 저장 등 실제 작업은 별도로 분리하여 구현



● Tool 구현 과정에서의 필요 사항

- Agent controller가 tool을 안정적으로 사용하려면 tool의 이름, 설명, 입력, 출력 형식이 명확해야 함

Tool name



- 호출할 함수 이름
- 기능을 드러내는 이름
- 예: `analyze_log`

Description



- 언제 사용할지 설명
- Agent controller가 참고
- 예: 오류 로그를 분석할 때 사용

Input / Output



- 함수 입력 인자
- 반환 dict 형식
- `status`, `result`, `message` 포함

Tool Interface 예시 (Python dict)

```
TOOL_REGISTRY = {  
    "analyze_log": {  
        "name": "analyze_log",  
        "description": "오류 로그에서 반복 알람을 분석한다.",  
        "function": analyze_log,  
    },  
    "calculate_metric": {  
        "name": "calculate_metric",  
        "description": "수치 지표를 계산한다.",  
        "function": calculate_metric,  
    },  
    "save_report": {  
        "name": "save_report",  
        "description": "분석 결과를 Markdown 파일로 저장한다.",  
        "function": save_report,  
    },  
}
```



3교시에서는 `name`, `description`, `function` 중심의 `tool registry`를 구성한다.

왜 중요한가?



- 1 Agent controller가 tool을 선택할 때
 - 어떤 상황에서 사용할 수 있는지 판단



- 2 입력/출력 형식을 알아야
 - 올바른 인자로 호출 가능
 - 결과를 prompt에 반영 가능



- 3 오류 형식을 알면
 - 실패 시 적절한 안내 가능
 - fallback 처리 가능

● Tool registry 정의

➤ Agent가 사용할 수 있는 tool 목록을 모아둔, 실행 가능한 기능 목록

① Registry의 역할



- 사용 가능한 tool 목록 관리
- tool 이름과 함수 연결
- tool description 관리

② 구성 요소



- **tool_name** : registry key (도구 이름)
- **description** : 언제 사용하는지 설명
- **function** : 실행할 실제 함수

③ 확장성



- 새로운 tool 추가 가능
- 4교시 search_manual() 등록 가능
- 5교시 Agent controller에서 재사용

TOOL_REGISTRY 예시 (Python dict)

```
TOOL_REGISTRY = {  
    "calculate_metric": {  
        "description": "수치 계산을 수행한다.",  
        "function": calculate_metric,  
    },  
    "analyze_log": {  
        "description": "오류 로그를 분석한다.",  
        "function": analyze_log,  
    },  
    "save_report": {  
        "description": "보고서를 Markdown 파일로 저장한다.",  
        "function": save_report,  
    },  
}
```

나중에 Agent가 어떻게 사용할까?



사용자 요청 입력

예) "오류 로그에서 반복 알람을 분석해줘"



Agent controller가 요청 해석

요청의 목적 파악



TOOL_REGISTRY 조회

사용 가능한 tool 목록과 설명 참조



적절한 tool 선택

예) analyze_log 선택



선택한 tool 실행

실행 결과를 받아옴



결과를 바탕으로 응답 생성

사용자에게 최종 답변 제공



핵심 포인트

Tool registry는 판단 로직이 아니라 실행 가능한 기능 목록이다.

3교시에서는 registry를 준비하고, 5교시에서 Agent controller가 이 목록을 조회해 적절한 tool을 선택·실행한다.

1. 함수 영역



- 업무 기능을 수행하는 Python 함수들
- 각 함수는 특정 작업을 담당

- calculate_metric()
- analyze_log()
- save_report()

2. Registry 영역



- 사용 가능한 tool 목록 관리
- tool 이름(key)과 설명 등록
- 실제 함수 객체 연결

3. 테스트 영역



- 각 tool 단독 실행 확인
- 입력/출력 형식 검증
- status, result, message 확인

Agent 연결 전에 검증 필수

tools.py

```
# 1) 함수 영역 : 업무 가능 함수
def calculate_metric(values, metric="average"):
    """수치 계산 수행"""
    ...
def analyze_log(log_data, code_key="error_code"):
    """오류 로그 분석 수행"""
    ...
def save_report(content, path="outputs/report.md"):
    """분석 결과를 Markdown 파일로 저장"""
    ...
```

```
# 2) Registry 영역 : 사용 가능한 tool 목록
TOOL_REGISTRY = {
    "calculate_metric": {
        "description": "수치 계산을 수행한다. 평균, 합계, 비율 등 지표를 산출한다.",
        "function": calculate_metric,
    },
    "analyze_log": {
        "description": "오류 로그에서 반복 알람과 오류 코드 빈도를 분석한다.",
        "function": analyze_log,
    },
    "save_report": {
        "description": "분석 결과를 Markdown(.md) 파일로 저장한다.",
        "function": save_report,
    },
}
```

```
# 3) 테스트 영역 : 각 tool 단독 실행 확인
if __name__ == "__main__":
    # 각 tool을 샘플 입력으로 실행하여 동작 확인
    ...
```

TOOL_REGISTRY 구조 예시

```
TOOL_REGISTRY = {
    "calculate_metric": {
        "description": "수치 계산을 수행한다. 평균, 합계, 비율 등 지표를 산출한다.",
        "function": calculate_metric,
    },
    "analyze_log": {
        "description": "오류 로그에서 반복 알람과 오류 코드 빈도를 분석한다.",
        "function": analyze_log,
    },
    "save_report": {
        "description": "분석 결과를 Markdown(.md) 파일로 저장한다.",
        "function": save_report,
    },
}
```

구성 항목	설명	예시
registry key (tool 이름)	도구를 식별하는 고유 이름 (Agent가 선택에 사용)	"analyze_log"
description (value)	언제, 어떤 상황에서 사용할지 설명	"오류 로그 분석"
function (value)	실제 실행할 Python 함수 객체	analyze_log

테스트 코드 예시 (각 tool 단독 실행)

```
if __name__ == "__main__":
    print("[TEST] calculate_metric()")
    print(calculate_metric([10, 20, 30]))

    print("[TEST] analyze_log()")
    print(analyze_log(sample_log))

    print("[TEST] save_report()")
    print(save_report("테스트 결과입니다.", "outputs/test_report.md"))
```

실행 결과 예시

```
[TEST] calculate_metric()
{'status': 'ok', 'result': {'average': 20.0, 'sum': 60, 'count': 3}, 'message': '계산 완료'}

[TEST] analyze_log()
{'status': 'ok', 'result': {'top_codes': [{'code': 'E001', 'count': 5}, {'code': 'E002', 'count': 3}], 'message': '분석 완료'}

[TEST] save_report()
{'status': 'ok', 'result': {'path': 'outputs/test_report.md'}, 'message': '파일 저장 완료'}
```



제공 코드 (강사 제공)

- tools.py skeleton 파일
- 함수 이름과 입력 구조 제공
- TOOL_REGISTRY 기본 형태 제공
- 테스트 코드 기본 위치 제공

→ 구조는 동일하게 유지



수강자 작업

- TODO 영역 구현
- 각 tool 단독 테스트
- TOOL_REGISTRY의 description과 function 연결 확인

→ 기능을 완성하고 검증



바이브코딩(GenAI) 활용

- 함수 내부 로직 구현 보조
- 오류 처리 코드 추가
- 반환 dict 형식 정리
- 테스트 입력 예시 생성

→ 코드 품질과 생산성 향상

tools.py skeleton (예시)

```
1 # tools.py skeleton (강사 제공)
2
3 # 1. 함수 영역 : 업무 기능 함수
4 def calculate_metric(values: list, metric: str) -> dict:
5     # TODO: 수치 계산 로직 구현
6     pass
7
8
9 def analyze_log(log_data: list[dict], code_key: str = "error_code") -> dict:
10    # TODO: 로그 분석 로직 구현
11    pass
12
13
14 def save_report(content: str, path: str = "outputs/report.md") -> dict:
15    # TODO: 파일 저장 로직 구현
16    pass
17
18 # 2. TOOL_REGISTRY 영역 : 사용 가능한 tool 목록
19 TOOL_REGISTRY = {
20     "calculate_metric": {
21         # TODO: description과 function 연결
22     },
23     "analyze_log": {
24         # TODO: description과 function 연결
25     },
26     "save_report": {
27         # TODO: description과 function 연결
28     },
29 }
30
31 # 3. 테스트 영역 : 각 tool 단독 실행 확인
32 if __name__ == "__main__":
33     # TODO: 각 tool 단독 실행 테스트 코드 작성
34     pass
```

TODO 가이드

각 함수 내부의 TODO 주석을 찾아
구현 코드를 작성하세요.

TODO: ~ 구현



구현할 내용

- 핵심 로직 구현
- 입력값 검증
- 예외 처리
- dict 형태로 결과 반환 (status, result 등 포함)

입력/출력 예시 (모든 tool 반환 dict)

calculate_metric(values, metric)
→ {"status": "ok", "result": 12.34,
"message": "계산 완료"}

analyze_log(log_data)
→ {"status": "ok", "summary": {...},
"message": "분석 완료"}

save_report(content, path)
→ {"status": "ok", "path": "...",
"message": "저장 완료"}

실습 진행 흐름



- 1 제공 skeleton 확인
파일 구조와 함수 시그니처,
TODO 위치 확인



- 2 TODO 구현
각 함수의 TODO 영역을
순서대로 구현



- 3 Tool 단독 테스트
각 tool을 직접 실행하여
입력/출력 결과 확인



- 4 TOOL_REGISTRY
연결 확인
• tool name (key)
• description
• function 객체



핵심 정리



tools.py는 세 영역으로 구성된다.

- 함수 영역
- TOOL_REGISTRY 영역
- 테스트 영역



각 tool은 Agent 연결 전에
단독 실행으로 검증한다.

- 상태(status) 확인
- 결과(result) 확인
- 메시지(message) 확인



4교시에는 search_manual()을 추가하여
문서 근거 검색으로 확장한다.

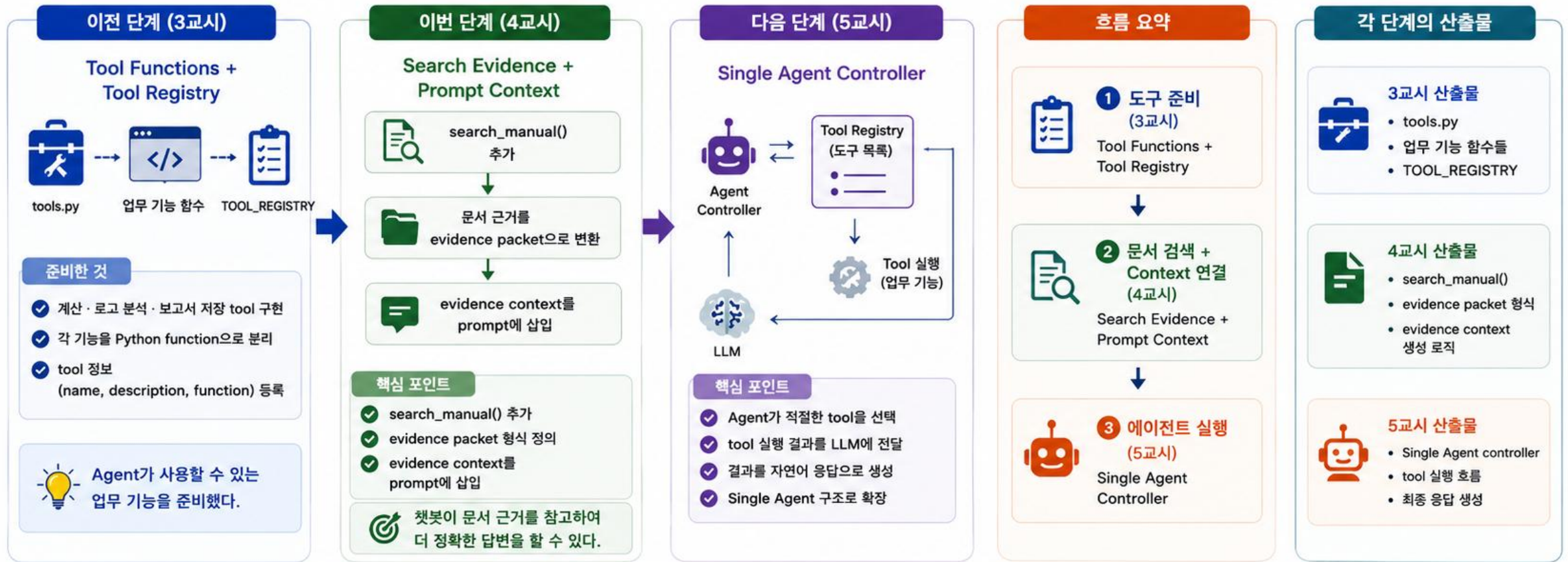
- 문서 검색 → 근거(evidence) 확보
- 보고서 생성과 연결



완성된 tool과 registry는
5교시에서 Agent controller가
선택하여 사용한다.

04

문서 검색 Tool과 Context 주입 구조



핵심 포인트

- 3교시: Agent가 사용할 수 있는 업무 tool을 준비했습니다.
- 4교시: 문서 근거가 필요한 질문을 위해 search_manual()을 추가하고, 반환된 evidence packet의 context를 LLM prompt에 넣는 구조를 이해합니다.
- 5교시: Agent가 tool과 evidence를 선택하고, 결과를 활용해 응답을 생성합니다.

완성된 tool과 registry는 5교시에서 Agent controller가 선택하여 사용합니다.

실습 기본 세트

chat_cli.py
사용자 입력 처리, 응답 출력

llm.py
local LLM 호출, assistant response 반환

messages list 기반 대화 구조 (system / user / assistant)

모든 단계에서 messages list를 유지하면서 대화를 이어갑니다.

업무형 agent는 일반 지식으로 추정하는 것이 아니라, 제공된 문서, 로그, 리포트에서 찾은 근거를 바탕으로 응답해야 함



A. 일반 LLM 응답의 한계

- 내부 지식에 의존
- 제공 문서 반영 어려움
- 근거 없는 답변 가능성 존재



B. 문서 근거 검색의 역할

- 관련 매뉴얼 내용 탐색
- 오류 코드 설명 확인
- 품질 이슈 메모 또는 리포트 근거 확보



C. 업무형 Agent의 요구

- 문서 기반 답변
- 출처 기반 설명
- 근거 부족 시 추정 금지



질문: E-104 알람 원인을 매뉴얼 기준으로 설명해줘

1) 일반 LLM 응답



일반 지식 사용

추정 가능

근거 불명확

E-104는 일반적으로 장비 이상과 관련될 수 있습니다...

(추정 기반, 신뢰 낮음)

2) Search Evidence 기반 응답



search_manual()
실행

관련 문서
조각 확인

evidence packet
생성

context를
prompt에 삽입

근거 기반
답변

매뉴얼 기준으로 E-104 알람은 모터 과부하 또는 센서 신호 불안정과 관련될 수 있습니다.

(근거 기반, 신뢰 높음)



출처: equipment_manual.txt

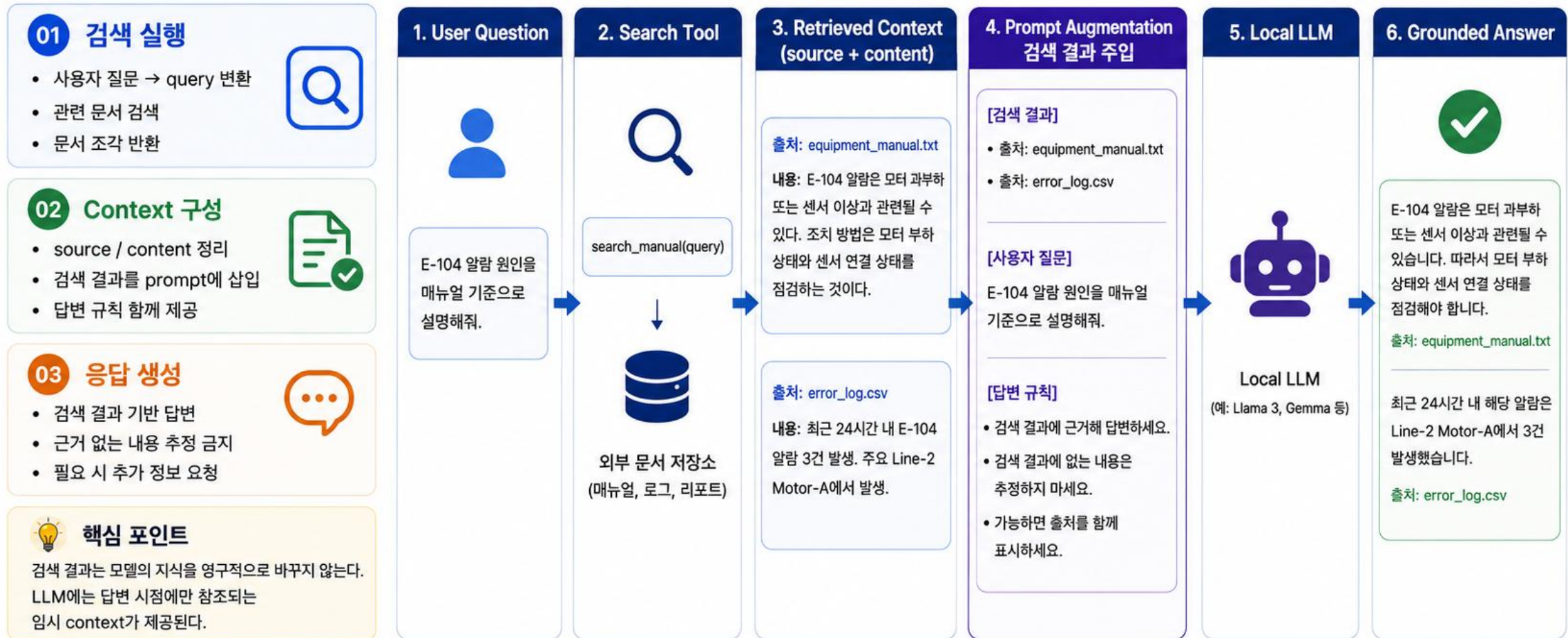
내용: E-104 알람은 모터 과부하 또는 센서 신호 불안정과 관련될 수 있다.



"매뉴얼 기준", '사내 문서 기준', '로그 기준'과 같은 요청은 먼저 search_manual()로 근거를 찾고, 그 결과를 evidence context로 넣어 답변해야 한다.

● Search tool의 결과와 context

- 검색 결과는 현재 질문에 답하기 위한 외부 근거 context로 prompt에 삽입
- 매뉴얼, 사내 문서, 로그 등 문서 근거가 필요한 질문에 대해 신뢰 가능한 답변 생성 가능



Evidence packet의 'context'는 사용자 질문·응답 규칙과 결합되어 augmented prompt로 LLM에 전달됨

01 근거 Context



- 관련 문서 조각
- 문서 출처
- 매칭 키워드 또는 검색 기준

02 사용자 질문



- 원래 질문 유지
- 검색 query로도 활용
- 답변 대상 명확화

03 응답 규칙



- 검색 근거에 기반해 답변
- 근거에 없는 내용은 추정 금지
- 출처가 있으면 함께 표시

Prompt 구성 예시

[검색 근거 (evidence context)]

출처: equipment_manual.txt

내용: E-104 알람은 모터 과부하 또는
센서 신호 불안정과 관련될 수 있다.

매칭 키워드: E-104, 모터, 센서

검색 기준: 매뉴얼에서 E-104 알람 원인 탐색

[사용자 질문]

E-104 알람 원인을 매뉴얼 기준으로 설명해줘.

[응답 규칙]

- 검색 근거에 기반하여 답변하세요.
- 근거에 없는 내용은 추정하지 마세요.
- 출처가 있으면 함께 표시하세요.



context가 비어 있을 경우
“근거 없음” 상태를 prompt에 명시한다.

build_augmented_prompt()



Augmented Prompt

사용자 질문
+
evidence context
+
응답 규칙



Local LLM



근거 기반 답변

E-104 알람은 모터
과부하 또는 센서 신호
불안정과 관련될 수
있습니다.

출처:
equipment_manual.txt

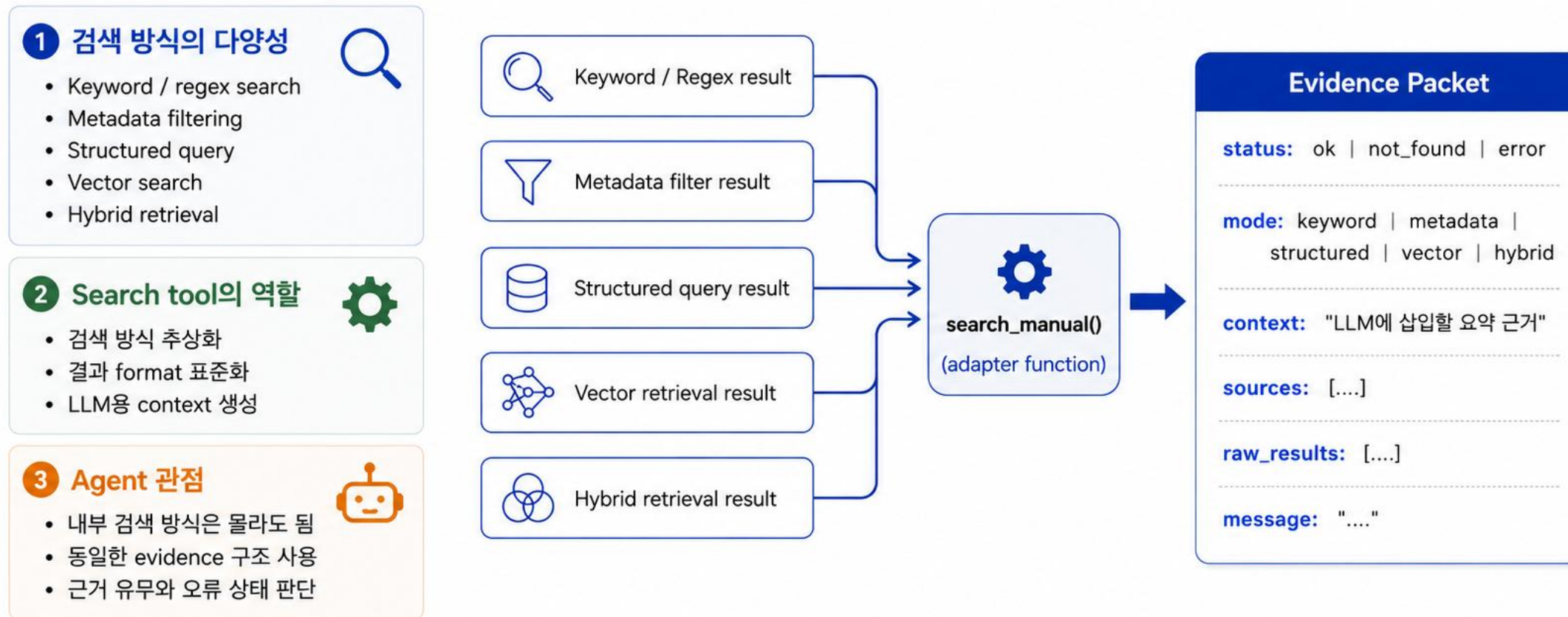


핵심 포인트

LLM에게 **evidence context**만 전달하는 것이 아니라,
그 근거를 어떻게 사용해야 하는지까지 함께 지시해야 **hallucination**을 줄일 수 있다.

● Search Tool의 역할

- 검색 방식(Keyword 중심, RAG 등)의 차이를 감추고 agent가 사용할 표준 evidence packet을 반환



● Evidence packet 구조화의 필요성

- 검색 결과는 단순 문자열이 아니라, status, mode, context, sources, results, has_context를 포함한 packet 형태로 관리하는 것이 안전함

01 상태 정보



- status : ok, not_found, error
- mode : keyword, vector, hybrid, none
- message : 검색 상태 설명

02 근거 정보



- context : LLM prompt에 넣을 근거 텍스트
- sources : 문서 출처 목록
- results : 세부 검색 결과

03 사용 기준



- has_context가 True이면 근거 기반 답변
- has_context가 False이면 “근거 없음” 상태를 prompt에 명시
- status로 정상, 미검색, 오류 상태 구분

Evidence Packet 구조 예시

```
evidence = {  
  "status": "ok",  
  "mode": "keyword",  
  "has_context": True,  
  "context": "[출처] equipment_manual.txt\n[내용]  
E-104 알람은 모터 과부하 또는 센서 신호 불안정과  
관련될 수 있습니다.",  
  "sources": ["equipment_manual.txt"],  
  "results": [  
    { "title": "E-104 알람 매뉴얼", "score": 0.92, "chunk_id": 3 },  
    { "title": "알람 코드 목록", "score": 0.87, "chunk_id": 7 }  
  ],  
  "message": "검색 결과를 사용했습니다."  
}
```



상태 정보

검색 상태와 방식



근거 Context

LLM에 넣을 근거 텍스트



출처 목록

참고한 문서 목록



세부 검색 결과

검색된 조각과 점수



메시지

검색 상태 설명



context가 비어 있으면 has_context는 False이며,
“근거 없음” 상태를 prompt에 명시하여 LLM이 근거 없이 추정하지 않도록 한다.

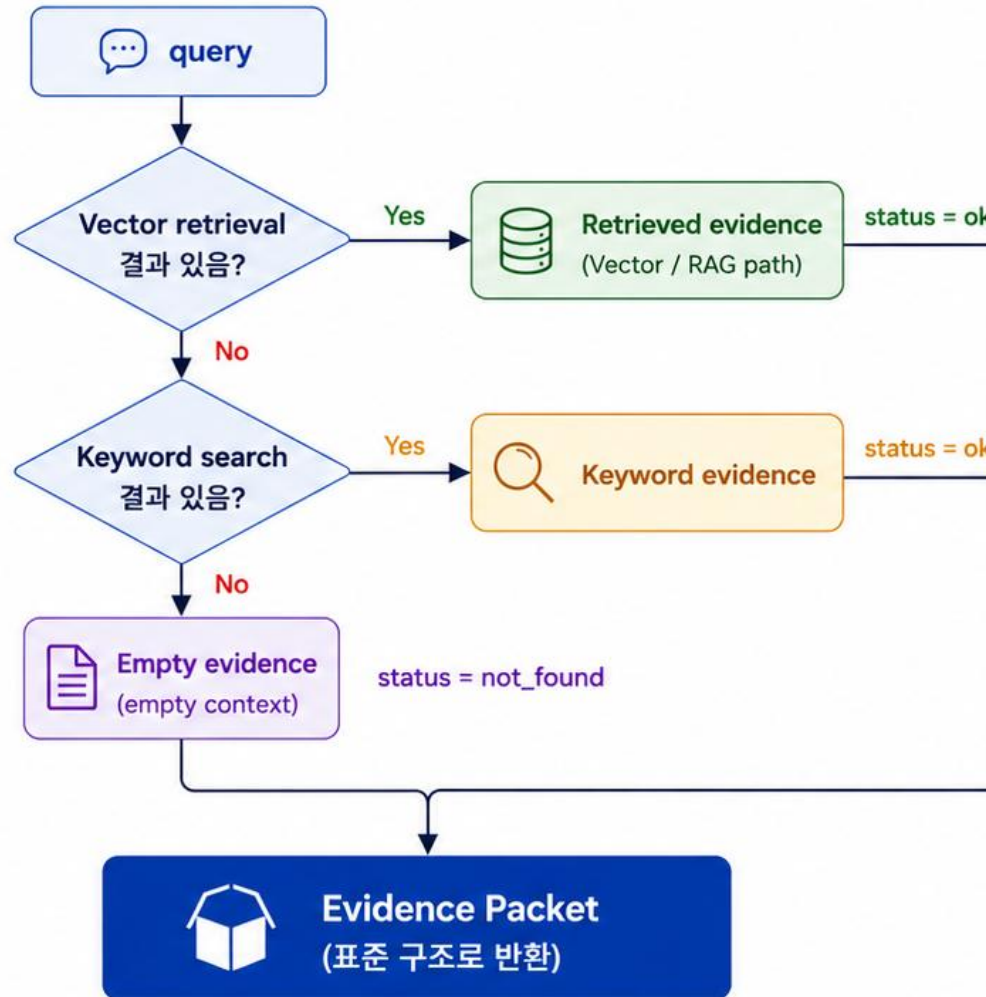
Retrieval 결과를 우선순위에 따라 확인하고, agent가 처리할 하나의 표준 evidence packet으로 표준화함

- 1

우선순위
 - Vector retrieval 결과가 있으면 retrieved evidence 사용
 - Keyword 결과가 있으면 keyword evidence 사용
 - 둘 다 없으면 empty evidence 반환
- 2

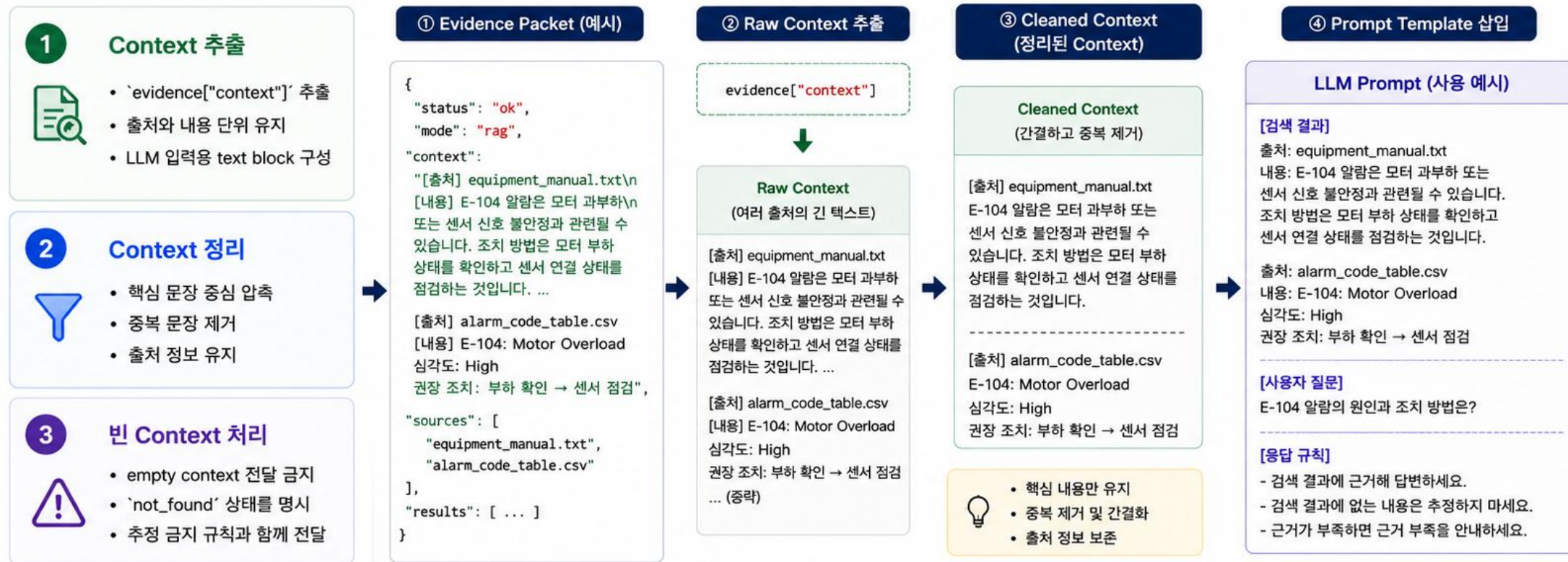
Fallback 구조
 - 검색 실패 시 오류를 감춤
 - empty context + not_found status 반환
 - status로 검색 실패 상태 표시
- 3

실습 기준
 - 검색 성능 구현은 선택
 - evidence packet 반환은 필수
 - dummy 또는 empty evidence로 흐름 확인 가능



Evidence Packet 구조	
	status ok not_found error
	mode vector keyword empty
	context (LLM에 삽입할 요약 근거)
	sources [문서 출처 목록]
	raw_results [원본 검색 결과]
	message (추가 정보 / 오류 설명)

Evidence packet의 'context'는 LLM이 답변할 때만 참조하는 임시 근거 자료 → 정리된 context block 형태로 prompt에 삽입



● 전체 코드 구조

- chat_cli.py: 실행 흐름을 관리 / search_manual()은 evidence packet을 반환
- LLM 호출에는 augmented prompt를 사용하되, message list에는 원래 질문과 응답만 저장



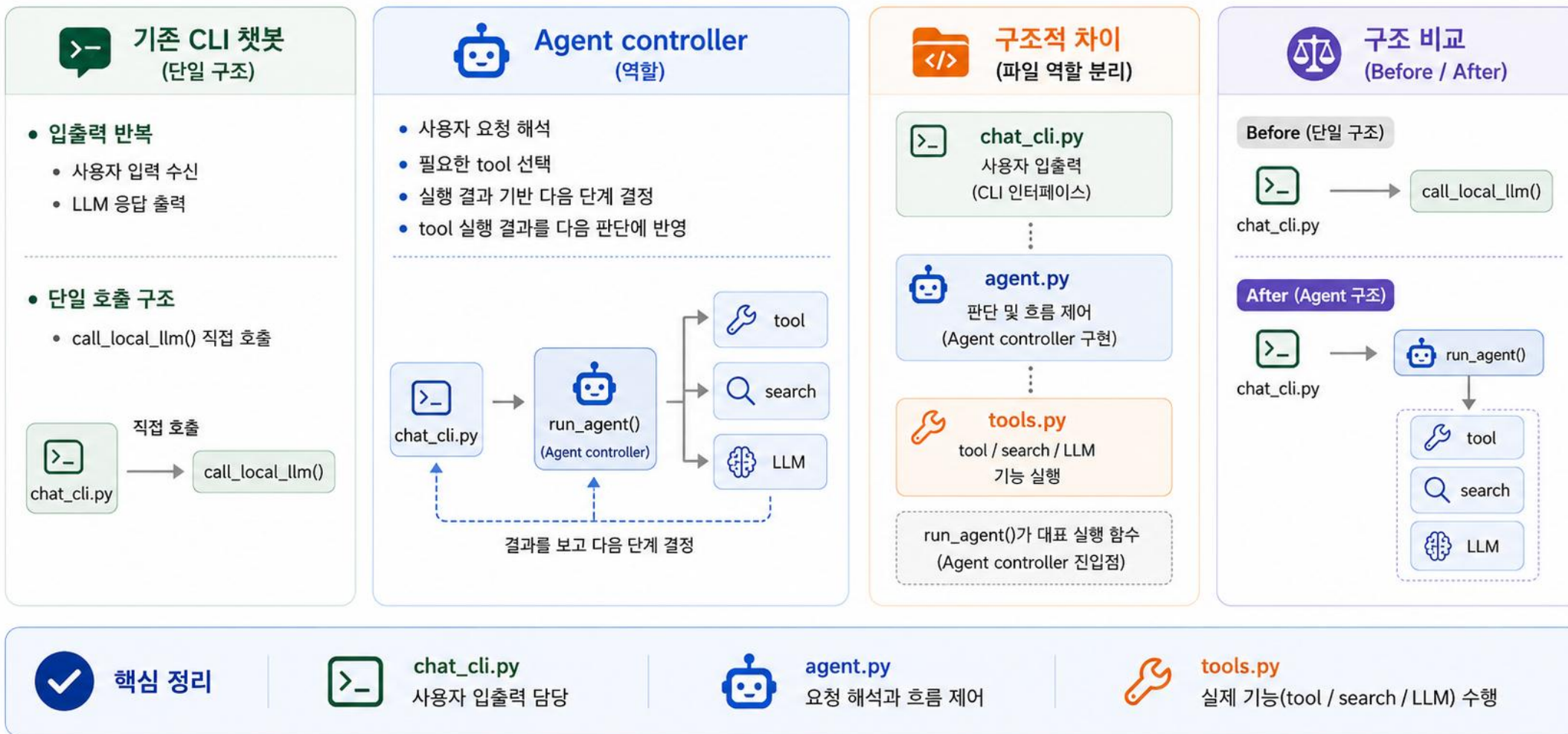
05

Single Agent Controller 구현



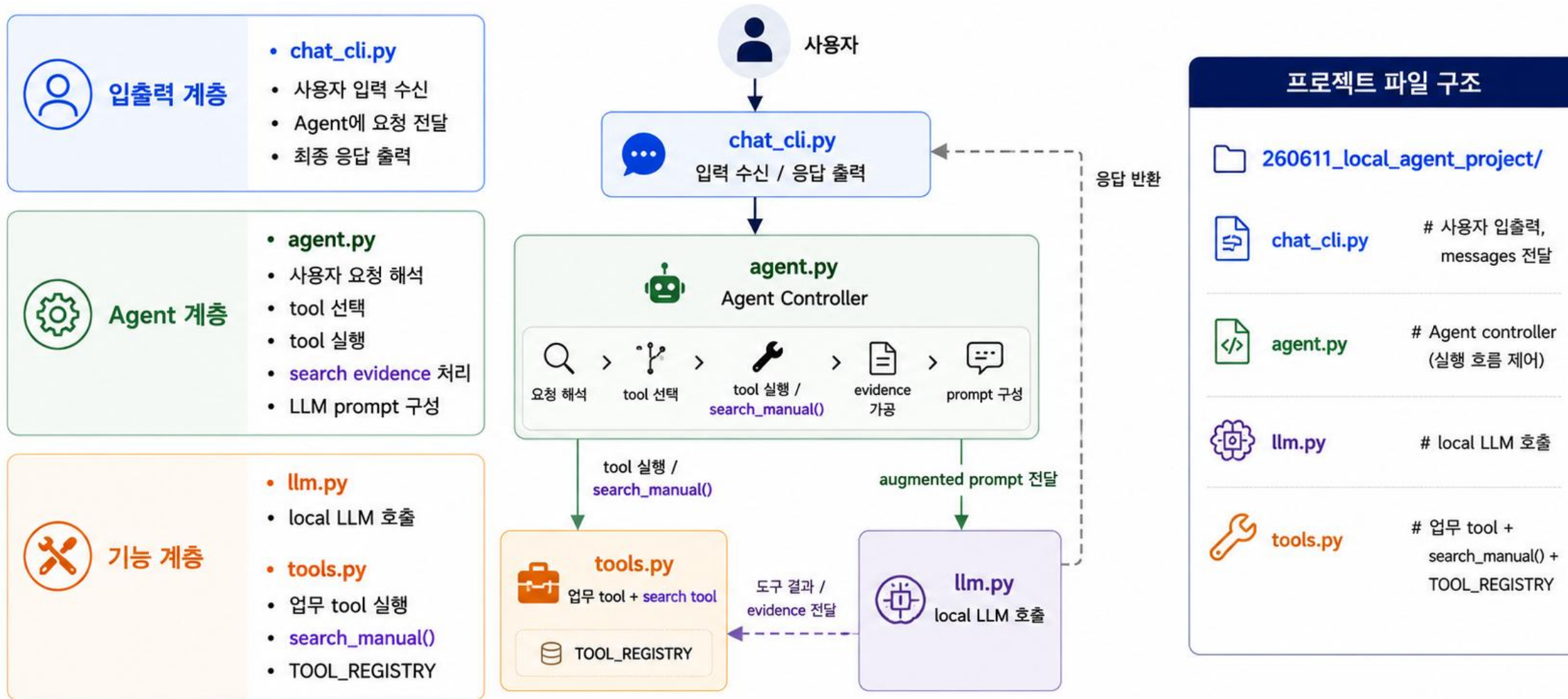
● Agent controller

- CLI 입출력과 실제 기능 수행 사이에서 요청 해석, tool 선택, 실행 흐름을 제어

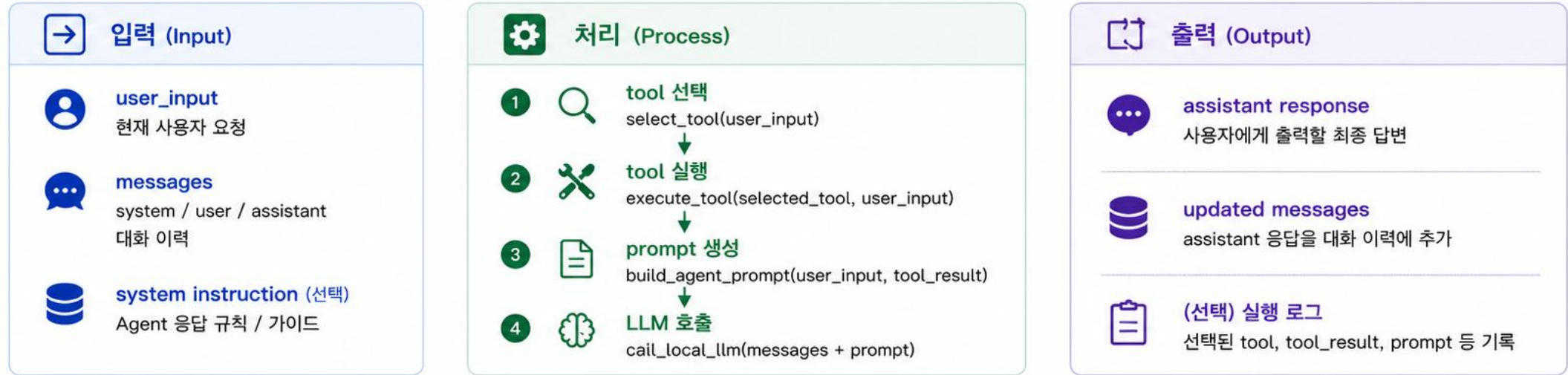


● 주요 변경점

- Agent.py를 추가하여 chat_cli.py, llm.py, tools.py의 tool registry와 search evidence 흐름을 연결



run_agent(): Agent의 중심함수로서, 사용자 입력을 해석하고 tool 실행 결과를 prompt로 구성해 LLM 응답을 생성



```
def run_agent(user_input: str, messages: list[dict]) -> str:  
    selected_tool = select_tool(user_input)  
    tool_result = execute_tool(selected_tool, user_input)  
    prompt = build_agent_prompt(user_input, tool_result)  
    response = call_local_llm(messages + [{"role": "user", "content": prompt}])  
    return response
```



함수 시그니처

run_agent(user_input: str, messages: list[dict]) -> str

- 입력: 현재 사용자 요청(user_input)과 대화 이력(messages)
- 출력: LLM이 생성한 최종 답변(response)

초기 실습: keyword rule로 tool 선택 흐름 고정 → LLM이 사용자 의도를 해석해 tool을 선택하도록 확장 가능

1 Rule 기반 선택 (keyword rule)

로그 분석 rule
키워드: 로그 / 알람 / 오류 → **analyze_log**

지표 계산 rule
키워드: 평균 / 합계 / 비율 → **calculate_metric**

보고서 저장 rule
키워드: 보고서 / 저장 / 파일 → **save_report**

문서 검색 rule
키워드: 매뉴얼 / 문서 / 근거 → **search_manual**

포인트 keyword rule은 단순하고 학습이 쉬워 디버깅과 구조 이해에 유리하다.

2 선택 결과 처리

tool name 반환
• 예: "analyze_log"

TOOL_REGISTRY 조회
• tool name과 실제 함수 매핑 확인
• 예: {"analyze_log": analyze_log, ...}

해당 tool 실행
• 선택된 함수 호출
• 실행 결과를 prompt 생성 단계로 전달

3 No tool (None) 처리

적절한 tool 없음
• rule에 매칭되지 않는 경우

일반 LLM 응답
• 단순 대화 또는 설명 요청 처리

지원 범위 안내
• 업무 tool 요청이지만 매칭 실패한 경우
• 지원 가능한 기능과 추가 입력 요청 안내

★ 확장 방향: LLM 기반 tool selection

✓ 사용자 의도를 LLM이 해석

✓ tool description을 참고해 tool 선택

✓ 선택 결과를 JSON 형태로 반환

User input + Tool descriptions

↓

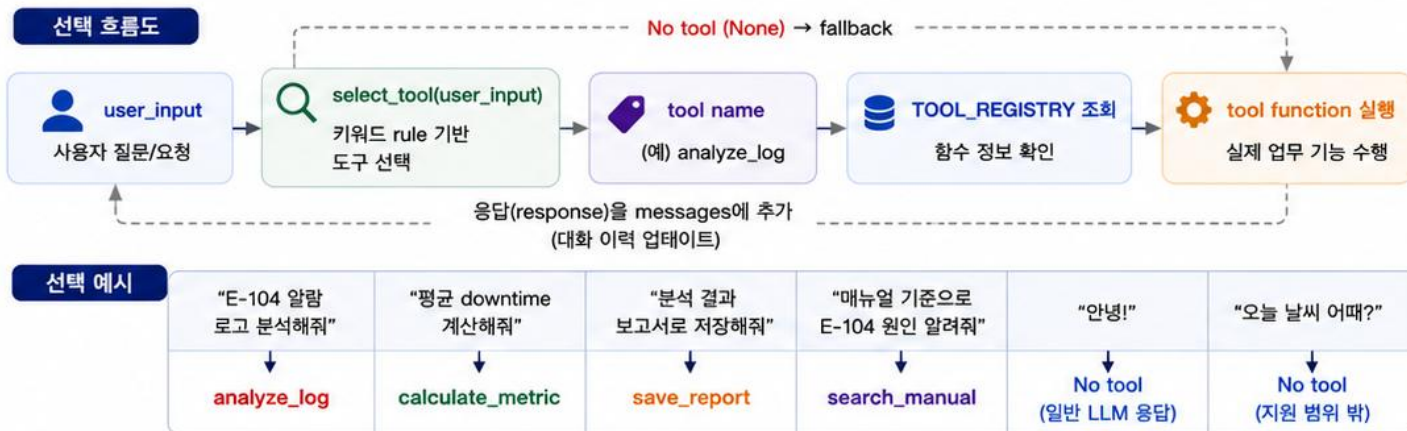
LLM tool selection

↓

```
{
  "tool": "search_manual",
  "reason": "문서 근거가 필요한 요청"
}
```

비교

- rule 기반: 구조 이해와 디버깅에 유리
- LLM 기반: 표현 다양성과 확장성에 유리



✓ 핵심 정리

keyword rule은 초기 실습에서 선택 흐름을 명확히 보여주기 위한 단순 정책이며, 이후에는 LLM이 사용자 의도와 tool description을 기반으로 tool을 선택하도록 확장할 수 있다.

Agent controller는 선택된 tool name을 이용해 TOOL_REGISTRY에서 실제 실행 함수를 찾아 호출



1. Registry 조회

- tool name 확인
- description 확인
- function 객체 추출



2. Tool 실행

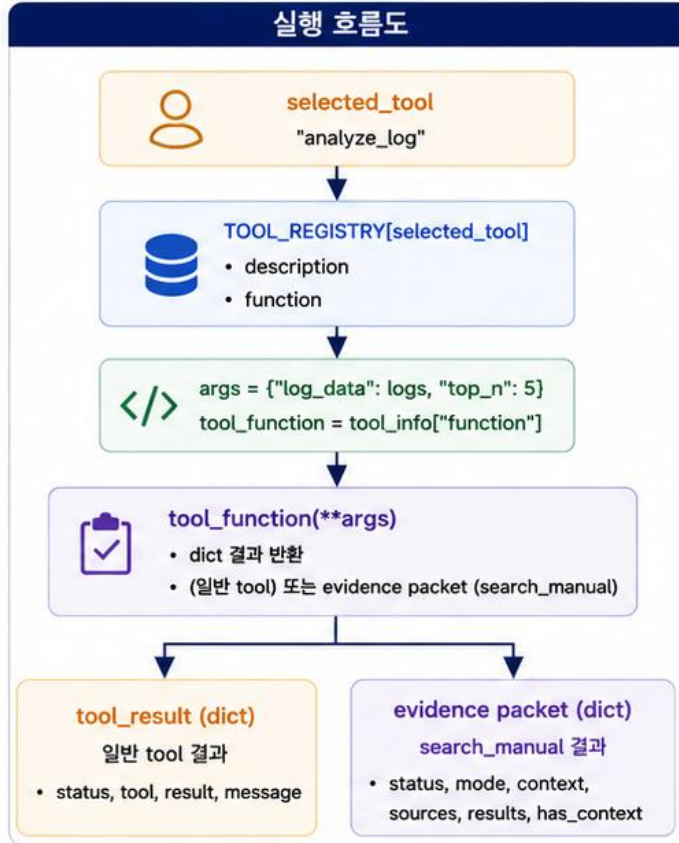
- 입력 인자 구성
- 함수 호출
- 결과 dict 반환



3. 실행 결과

- **status**: 실행 여부
- **tool**: 실행된 tool 이름
- **result** 또는 **summary**: 실행 결과
- **message**: 추가 설명

실행 흐름도



코드 예시

```
selected_tool = "analyze_log"
tool_info = TOOL_REGISTRY[selected_tool]

# 1) Registry 조회
tool_desc = tool_info["description"]
tool_function = tool_info["function"]

# 2) Tool 실행
args = {"log_data": logs, "top_n": 5}
tool_result = tool_function(**args)

# 3) 실행 결과 사용
print(tool_result)
```

TOOL_REGISTRY 예시 (tools.py)

키 (tool name)	설명 (description)	함수 (function)	반환 타입
analyze_log	오류 로그 분석 및 반복 알람 확인	analyze_log	dict
calculate_metric	평균, 합계, 비율 등 수치 계산	calculate_metric	dict
save_report	분석 결과를 Markdown 보고서로 저장	save_report	dict
search_manual	매뉴얼/문서 검색 및 근거 확보	search_manual	evidence packet



핵심 정리



Tool registry는 판단 로직이 아니라, 선택된 tool name으로 실제 실행 함수를 찾기 위한 연결 테이블이다.



Agent controller가 tool을 선택하고, registry는 실행 대상을 안전하게 조회하여 tool을 호출한다.

Tool 실행 결과는 그대로 출력하지 않고, 사용자 질문과 함께 LLM prompt에 넣어 자연어 답변으로 변환

1. Tool result

- Python dict 형태
- 계산 결과 또는 분석 결과 포함
- 코드가 다루기 쉬운 구조

예시 (analyze_log 결과)

```
{
  "status": "ok",
  "tool": "analyze_log",
  "summary": {
    "total_logs": 120,
    "top_error_codes": [{"code": "E-104", "count": 45}, {"code": "E-207", "count": 18}],
    "repeated_error_codes": [{"code": "E-104", "count": 45}]
  },
  "message": "로그 분석이 완료되었습니다."
}
```

2. Prompt 변환

- 사용자 요청 유지
- tool 실행 결과를 LLM이 이해할 수 있는 형태로 삽입
- 답변 형식과 규칙을 명시

Prompt 구성 예시

[사용자 요청]

E-104 알람의 주요 원인과 대응 조치를 분석해줘.

[Tool 실행 결과]

{format_tool_result(tool_result)} ← dict를 LLM이 읽기 쉬운 텍스트(JSON)로 변환

[응답 규칙]

- tool 결과를 바탕으로 답변하세요.
- 없는 내용은 추가로 만들지 마세요.
- 필요한 경우 후속 확인 항목을 제시하세요.

3. LLM 응답

- 결과 요약
- 의미 해석
- 후속 확인 항목 제시

예시 응답

분석 결과, E-104 알람이 총 45건으로 가장 많이 발생했습니다.
주요 원인은 다음과 같습니다.

- 메모리 과부하로 인한 재발
- 센서 신호 불안정

다음 조치를 권장합니다.

1. 모듈 부하 및 온도 상태 점검
2. 엔시 내결 상태 및 노이즈 확인
3. 동일 알람이 지속되면 장비 점검 수행

다음 항목을 추가로 확인하세요.

- 모터 전류 로그
- 센서 원시 신호 데이터



코드 예시

```
prompt = f"""
[사용자 요청]
{user_input}

[Tool 실행 결과]
{json.dumps(tool_result, ensure_ascii=False, indent=2)}

[응답 규칙]
- tool 결과를 바탕으로 답변하세요.
- 없는 내용은 추가로 만들지 마세요.
- 필요한 경우 후속 확인 항목을 제시하세요.
"""
```

중요 원칙

- ✓ LLM 호출용 프롬프트에만 tool 결과를 포함합니다.
- messages list에는 원래 질문과 최종 응답만 저장합니다.

핵심 정리

- Tool result는 코드가 다루기 쉬운 구조화 데이터(dict)이다.
- LLM은 이 데이터를 사용자 질문과 함께 해석하여 업무형 자연어 답변으로 변환한다.
- Prompt에는 "tool 결과에 없는 내용은 만들지 말라"는 규칙을 함께 넣어 신뢰도를 높인다.

search_manual()의 결과는 tool_result가 아니라 evidence packet으로 처리합니다.

● Fallback 처리

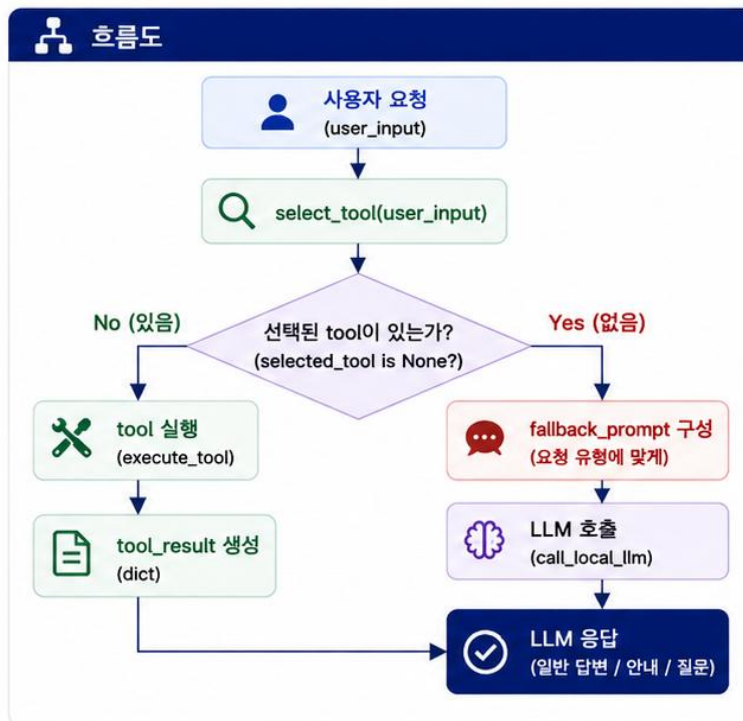
- 등록된 tool로 처리할 수 없는 요청은 억지로 실행하지 않음
- 요청 유형에 따라 일반 LLM 응답, 추가 정보 요청, 혹은 지원 범위 안내로 전환

1. No tool 상황

- TOOL_REGISTRY에 적절한 tool 없음**
 - 요청을 처리할 수 있는 tool 부재
- 필요한 capability 미등록**
 - 외부 데이터 / API / 검색 도구 필요
- 입력 정보 부족**
 - 필요한 파라미터나 맥락이 부족

핵심 관점

No tool 처리는 실패가 아니라, Agent가 수행 가능한 범위를 벗어난 요청을 안전하게 처리하는 정상 분기이다.



2. 처리 방식

- 일반 LLM 응답**
 - 단순 대화 또는 개념 설명 처리
- 추가 정보 요청**
 - 입력 정보가 부족한 경우
 - 필요한 정보(파라미터, 기간 등)를 요청
- 지원 범위 안내**
 - 현재 등록된 tool로 처리 불가능한 경우
 - 지원 가능한 기능과 대안 안내

3. 설계 원칙

- 비슷한 tool 강제 실행 금지**
 - 잘못된 결과와 오류 확산 방지
- 미지원 기능 명시**
 - hallucination 방지와 신뢰성 확보
- 반복 요청 기록**
 - 신규 tool 후보로 관리 및 개선에 활용

선택 예시

"E-104 알람 로그 분석해줘"	"평균 downtime 계산해줘"	"분석 결과 보고서로 저장해줘"	"매뉴얼 기준으로 E-104 원인 알려줘"	"안녕!"	"오늘 날씨 어때?"
analyze_log	calculate_metric	save_report	search_manual	No tool (일반 LLM 응답)	No tool (지원 범위 밖)

확장 방향: LLM 기반 tool selection

User input + Tool descriptions (도구 목록/설명) → LLM이 사용자 의도 해석 및 적절한 tool 선택 → 선택 결과(JSON) 예시

```

{
  "tool": "search_manual",
  "reason": "문서 근거가 필요"
}
    
```

● 핵심 동작

- 문서 검색이 필요한 요청 → search_manual() 결과를 evidence packet으로 받고, 이를 search-augmented prompt로 변환해 LLM에 전달



1. Search tool 실행

- 사용자 질문을 그대로 query로 사용
- search_manual() 호출
- evidence packet 반환



2. Prompt 구성

- evidence context 삽입
- 사용자 질문 유지
- 근거 사용 규칙 포함



3. 응답 생성

- has_context = true → 근거 기반 답변 생성
- has_context = false → 근거 부족 안내
- 출처(문서명 / section) 표시

흐름도



예시 1. evidence packet (search_manual 결과)

```
{
  "status": "ok",
  "mode": "search_manual",
  "has_context": true,
  "context": "E-104는 모터 과열 경고입니다. \n온도 센서 값이 기준을 초과하면 알람이 발생합니다.",
  "sources": [
    { "doc_id": "manual_01", "title": "E-104 알람 가이드", "section": 3, "score": 0.92 },
    { "doc_id": "manual_03", "title": "모터 과열 진단", "section": 1, "score": 0.81 }
  ],
  "raw_results": {
    "documents": [
      { "doc_id": "manual_01", "title": "E-104 알람 가이드", "score": 0.92 },
      { "doc_id": "manual_03", "title": "모터 과열 진단", "score": 0.81 }
    ],
    "sections": [
      { "doc_id": "manual_01", "section": 3, "text": "E-104는 모터 과열 경고입니다..." },
      { "doc_id": "manual_03", "section": 1, "text": "온도 센서 값이 기준을 초과하면..." }
    ]
  },
  "message": "검색 결과를 가져왔습니다. context를 사용해 답변하세요."
}
```

예시 2. Search-Augmented Prompt (LLM에 전달되는 프롬프트)

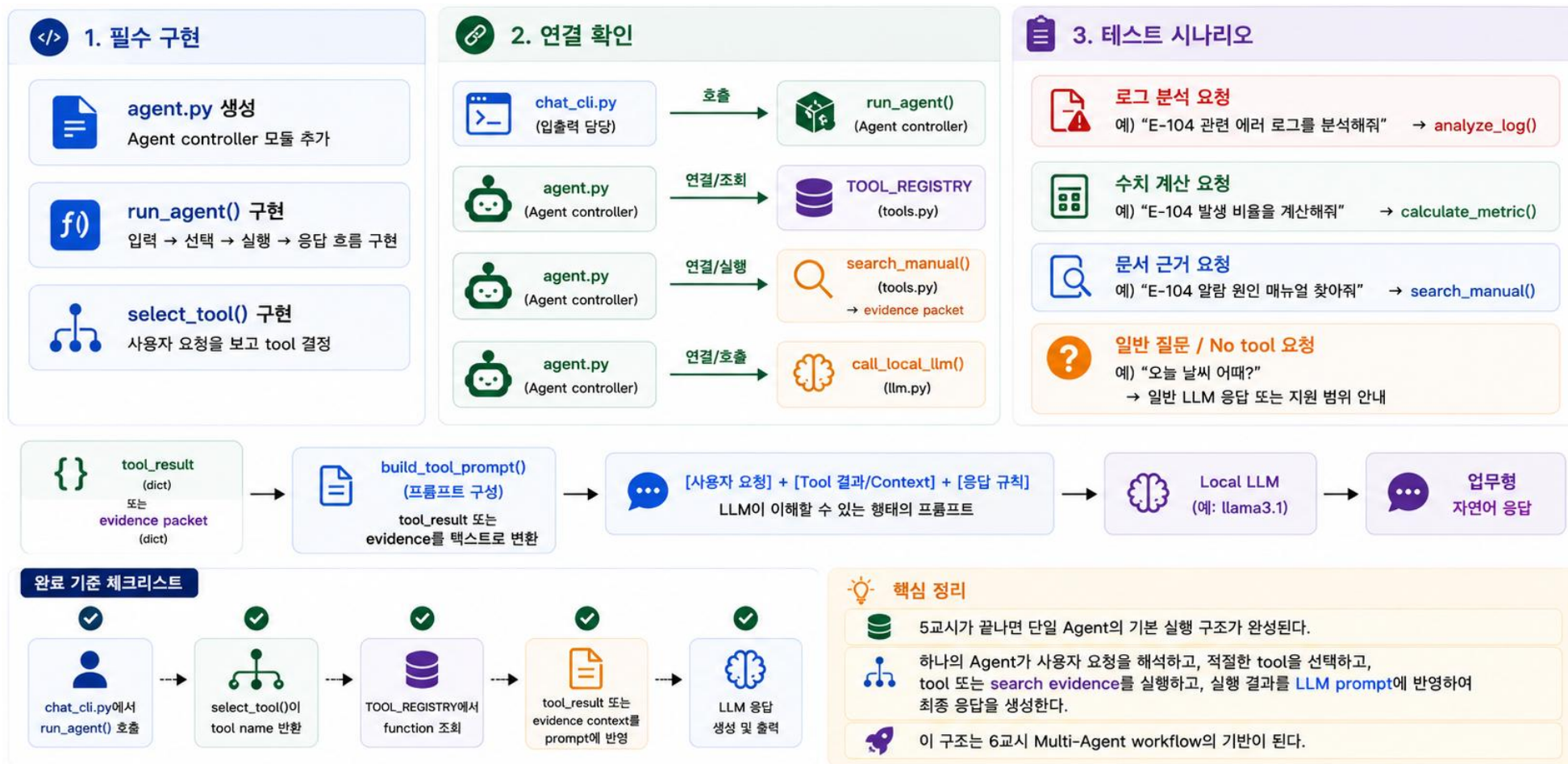
```
[사용자 질문]
E-104 알람 원인과 대응 방법은?

[검색 근거 Context block]
- 출처: manual_01, section 3
  내용: E-104는 모터 과열 경고입니다.
- 출처: manual_03, section 1
  내용: 온도 센서 값이 기준을 초과하면 알람이 발생합니다.

[응답 규칙]
- 위의 근거(Context)를 기반으로 답변하세요.
- 근거에 없는 내용은 추가로 생성하지 마세요.
- 가능한 한 문서명과 section 번호를 출처로 표시하세요.
- 추가 확인이 필요한 항목이 있으면 함께 제시하세요.
```

● 핵심 구현 요소

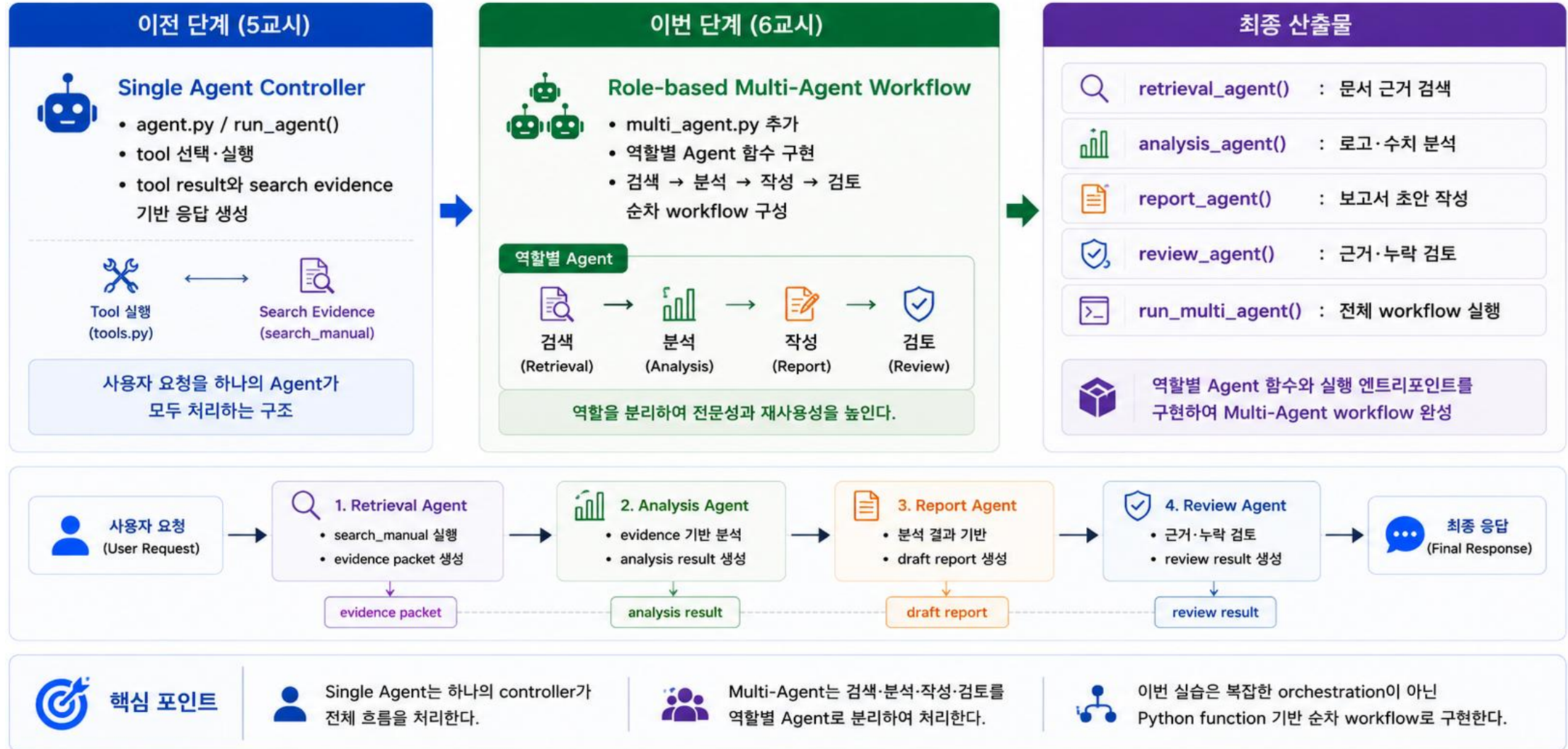
- Agent controller는 사용자 요청에 따라 tool 선택
- Tool result 혹은 search evidence를 LLM 응답 생성에 반영



06

Multi-Agent 소개와 간단한 역할 Workflow 구현

Single agent 구조를 역할에 따라 분리하여 role-based multi-agent workflow로 확장



하나의 판단, 하나의 실행 결과로 끝나는 요청 → single agent로 충분함



1. 단일 요청 유형

- 일반 설명: “E-104가 뭐야?”
- 단일 분석: “이 로그에서 가장 많이 나온 오류 코드는?”
- 단일 계산: “평균 downtime 계산해줘”



2. 단일 실행 구조

- 하나의 tool 선택
- 하나의 실행 결과 반환
- 추가 역할 분리 없음



3. Single Agent의 장점

- 구현 단순
- 디버깅 용이
- 초기 실습에 적합



User Request
(사용자 요청)



Single Agent
(단일 Agent)



Tool 실행 / LLM 응답
(단일 처리 경로)



Final Response
(최종 응답)



한계

- 여러 tool을 순차적으로 조합해야 하면 복잡도 증가
- 서로 다른 역할의 판단이 필요하면 구조가 비대해짐
- 계획-검색-분석-검토가 모두 필요한 요청은 Multi-Agent가 더 적합



그러나 여러 단계의 계획, 검색, 분석,
검토가 함께 필요한 요청은
Multi-Agent 구조가 더 적합하다.

하나의 요청을 처리하기 위해 여러 작업(검색, 분석, 작성, 검토)이 모두 필요 → multi-agent 구조가 유리



1. 복합 요청 (예시)

- “E-104 알람이 반복 발생한 원인을 근거와 함께 분석해줘”
- “매뉴얼 근거와 로그 분석 결과를 함께 반영해줘”
- “분석 결과를 조치 보고서 형식으로 정리하고 검토해줘”



2. 필요 작업

- ✓ 문서 근거 검색
- ✓ 로그·수치 분석
- ✓ 원인 후보 정리
- ✓ 보고서 작성
- ✓ 근거·누락 검토



3. Multi-Agent 이점

- ✓ 역할별 책임 분리
- ✓ 중간 결과 추적 가능
- ✓ 검토 단계 독립화

복합 업무 요청 처리 흐름 (Multi-Agent 구조)



검색 Agent
(Search Agent)

매뉴얼, 절차서 등에서
관련 근거 확보

evidence packet
생성



분석 Agent
(Analysis Agent)

오류 로그, 수치 데이터 분석
반복 패턴 및 이상 징후 확인



작성 Agent
(Writer Agent)

분석 결과와 근거를 기반으로
원인 후보 및 조치안 작성



검토 Agent
(Review Agent)

근거 연결성, 누락 항목,
조치안 타당성 점검



최종 응답
(Final Response)

검토 완료된 최종 보고서
원인 후보 / 근거 / 조치 권고안



따라서 Multi-Agent 구조에서는 각 Agent의 **역할, 입력, 출력, 전달 정보**를 명확히 정의해야 한다.

Single agent: 하나의 controller가 전체 흐름을 처리 / Multi-agent: 역할별 agent가 중간 산출물을 넘기며 협업



Single Agent

- 하나의 run_agent()
- 하나의 controller가 전체 흐름 제어
- 구현은 단순하지만 분기와 책임 증가



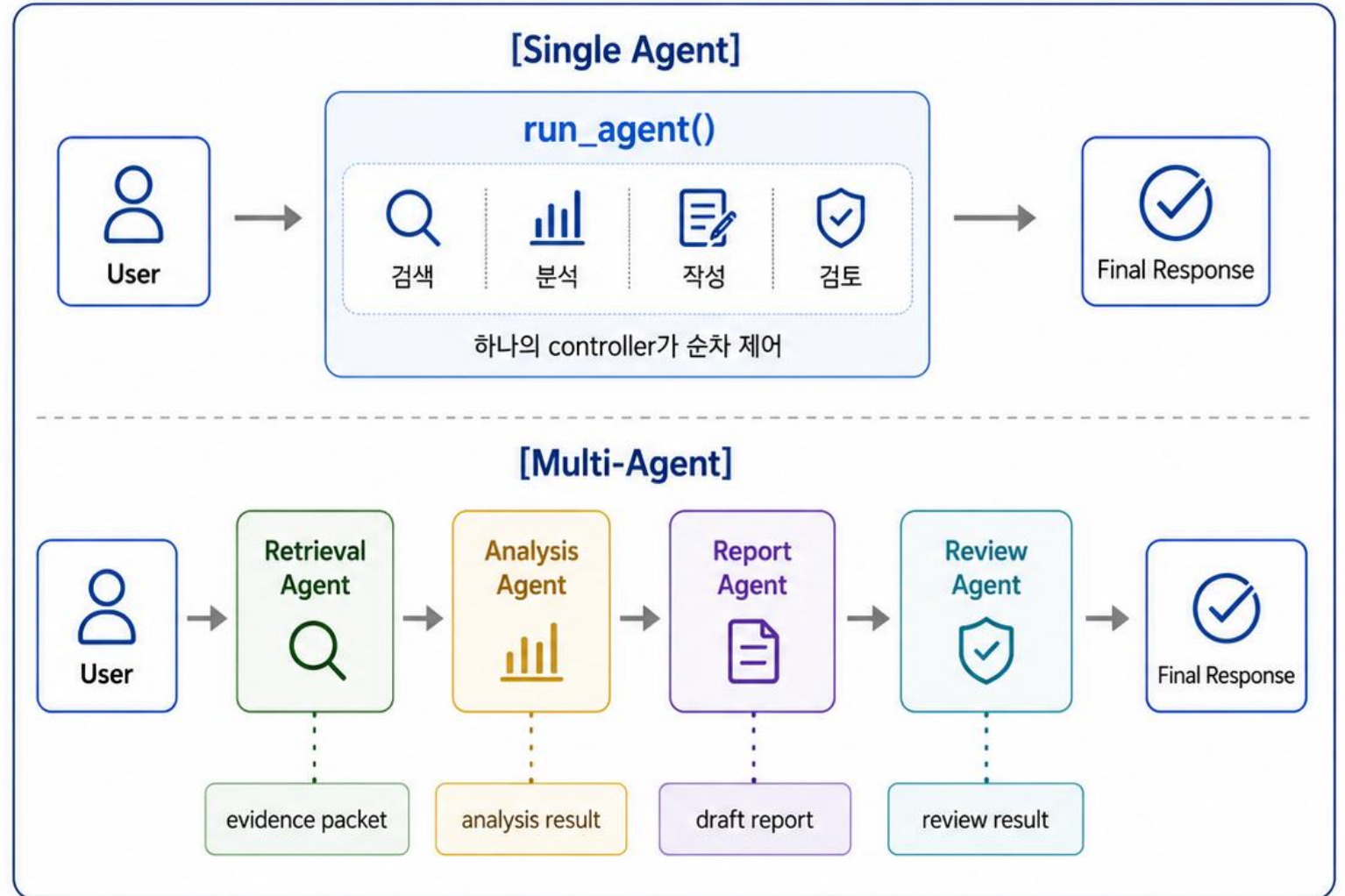
Multi-Agent

- 역할별 Agent 함수
- evidence packet, analysis result 등 단계별 생성
- 중간 산출물 확인 가능

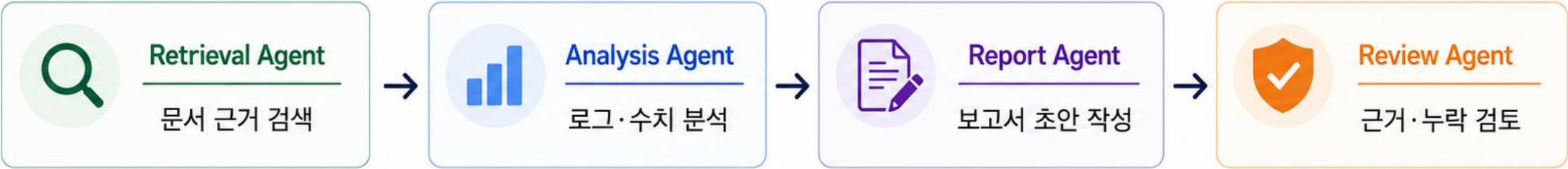


핵심 차이

- 핵심은 책임 분리 방식
- workflow와 입출력 계약의 차이
- 여러 LLM이 아니라 역할별 함수로 구현 가능



각 agent 함수는 하나의 역할만 담당하고, 다음 단계가 사용할 수 있는 구조화된 결과를 반환



Agent 함수	입력 (Input)	처리 (Process)	출력 (Output)
Retrieval Agent	user input	search_manual()	evidence packet
Analysis Agent	user input, evidence packet	analyze_log(), calculate_metric()	analysis result
Report Agent	evidence packet, analysis result	draft report 생성	draft report
Review Agent	draft report	checklist 기반 점검	review result

역할을 나눌 때는 “무엇을 입력받고, 무엇을 반환하는가”가 중요합니다.
출력이 명확하지 않은 Agent 함수는 다음 단계와 연결하기 어렵습니다.

구현 요소

1



역할 함수

- retrieval_agent()
- analysis_agent()
- report_agent()
- review_agent()

2



Workflow 함수

- run_multi_agent()
- 역할별 함수 순차 실행
- 최종 응답 반환

3



기존 코드 재사용

- **tools.py**: search_manual(), analyze_log(), calculate_metric()
- **llm.py**: call_local_llm()

프로젝트 구조

260611_local_agent_project/

chat_cli.py
llm.py
tools.py

multi_agent.py # 6교시 workflow controller

outputs/ # 선택: 보고서 저장 위치

최종 실행 흐름 제어



agent.py는 5교시 Single Agent 실습용
6교시 최종 실행은 multi_agent.py 중심

</> multi_agent.py 내부 구조

multi_agent.py

retrieval_agent()
analysis_agent()
report_agent()
review_agent()
run_multi_agent()



핵심 정리

6교시에서는 **multi_agent.py**가 전체 **workflow controller** 역할을 담당한다.

각 Agent 함수는 중간 산출물을 반환하고, **run_multi_agent()**가 이를 순차적으로 연결한다.

제공되는 multi_agent.py skeleton에 대하여, 역할별 TODO를 채우는 방식으로 진행

실습 구성

1

제공 코드



- multi_agent.py skeleton
- 함수 이름과 입력/출력 구조
- sample data

2

수강자 TODO



- retrieval_agent() TODO
- analysis_agent() TODO
- report_agent() TODO
- review_agent() TODO
- run_multi_agent() 전체 실행 확인

3

실습 원칙



- 파일 구조와 함수 이름은 고정
- 복잡한 orchestration 제외
- LangChain / LangGraph 사용 제외
- Python function workflow로 구현

TODO

retrieval 결과 반환
analysis 결과 구성
report text 작성
review checklist 작성

실습 흐름

1



multi_agent.py
skeleton 제공

2



TODO 구현

3



각 Agent 함수
단독 테스트

4



run_multi_agent()
단체 실행

바이브코딩 활용



함수 내부 구현 보조



오류 처리 아이디어 보완



중간 산출물 형식 정리

전체 구조 생성이 아닌
구현과 테스트 보조에 활용

감사합니다